

DEVELOPER GUIDE

The Dyon Developer Guide

Building agentic digital twins in Python

Twelve chapters · written to be read in order, then kept as a reference

Contents

Preface	4
The one rule	4
01 — The Mental Model	5
What a digital twin is in Dyon	5
The four layers	5
The four things you write	6
How the layers run, and how they talk	7
Configuration is the single source of truth	8
What the framework owns, and what you own	8
02 — Your First Twin	10
Step 1 — Scaffold the project	10
Step 2 — Start the infrastructure	10
Step 3 — Wait until everything is ready	11
Step 4 — Define the twin	11
Step 5 — Run	13
Step 6 — Drive it with the simulator	13
Step 7 — See where the data went	14
03 — Configuration and Sensors	16
Describing the asset	16
The anatomy of a sensor field	16
What the configuration computes for you	17
Connecting to infrastructure	18
Dev mode and production mode	18
Running several twins side by side	19
The sub-config reference	19
04 — The Data Layer	21
Why four stores instead of one	21
The TelemetryRouter: the way in	22
The DataManagementPipeline: from raw to useful	22
How the layers above read data	23
What ends up in InfluxDB	23
When the data is text: TextIngestor	23
When interactions have a beginning and end: sessions	24
When decisions must be auditable: ProvenanceLog	25
When you need SQL: PostgresAdapter	25
Replacing a backend	26
05 — The Simulation and Model Layer	27
Three kinds of model	27
The ModelRunner	27
Writing a physics model	28
Writing a surrogate model	29
Writing any other model	29
Running several models at once	30
The forecaster	30
Where the data comes from	30
06 — The Services Layer	32
Eclipse Ditto: a standard place to read twin state	32

The FastAPI web API	33
Streaming with Server-Sent Events	35
07 — The Agent Layer: Control	36
The ThresholdRuleEngine and its state machine	36
Custom rules	37
Richer assets: the MultiStateFSMRuleEngine	38
Continuous control: the PIDController	39
Versioned rules: the RuleRepository	40
08 — The Agent Layer: Reasoning	42
The knowledge graph	42
The diagnostic agent	44
Running agents continuously: the MultiAgentSystem	46
Choosing and skipping pieces	47
09 — The Agent Layer: Governance	48
The loop, step by step	48
Orient with rules: the GoalPlanner	48
Decide: the safety hierarchy	49
Wiring the loop together	50
Orient with an LLM: the AutonomousOverseer	50
Control with a learned policy: RL	51
When you cannot write the reward	53
Escalating to humans	53
10 — Connectors and Collection Twins	54
Connectors: how twins reach each other	54
Collection twins: many twins as one	55
Running several collections together	57
11 — A Full Worked Example	59
twin.py	59
run.py	62
simulate.py	62
.env	63
Running it, and what you will see	63
Adapting it to your asset	64
12 — Learning from Demonstration	65
Describing the observation: FeatureSpec	65
The common data format: Demonstrations	66
Copying the expert: behavioural cloning	66
Recovering the reward: AIRL	67
The whole flow at once: the skill-transfer pipeline	67
Deploying a learned policy	69
Which method to reach for	69
13 — Visualization	70
The one-line dashboard	70
What you get for free	71
How the dashboard is laid out	71
The specification is the contract	72
Live updates come from the event bus	72
The conversational interface	72
Watching the agents work	73
The 3D viewport	73
Combined twins: one dashboard for many	74
Optional backends	75

Where this leaves you 76

Preface

This is the complete manual for building a digital twin with Dyon. The thirteen chapters are written to be read in order the first time — each builds on the vocabulary and the layers introduced before it — and to serve as a reference afterwards.

The arc is simple. Chapters 1–3 give you the mental model, a first running twin, and the configuration that drives everything. Chapters 4–9 then walk up the four-layer architecture, from raw data at the bottom to agentic decision-making at the top: one chapter each for the Data, Simulation and Model, and Services layers, then three for the Agent layer, whose control, reasoning, and governance tiers are substantial enough to deserve a chapter apiece. Chapter 10 steps outside a single twin to how twins connect and combine. Chapter 11 assembles a full twin end to end, chapter 12 covers teaching a twin from demonstrations when you cannot write its reward by hand, and chapter 13 puts a live dashboard on the whole thing.

The one rule

You never edit anything inside the `dyon/` package. Everything you write — your config, your twin class, your models — lives in your own project folder beside it. The framework provides the mechanism; you provide the domain. Keep that line clear and the rest of the guide follows naturally.

01 — The Mental Model

This chapter has no code to run. Its job is to give you the vocabulary and the shape of the framework, so that when you start building in the next chapter every piece has somewhere to fit. Read it once, slowly. Everything afterwards refers back to the ideas introduced here.

What a digital twin is in Dyon

A digital twin is a long-running Python program that mirrors one real-world asset. “Asset” is deliberately vague: it might be a centrifugal pump, a wind turbine, an HVAC unit, or a delivery fleet. The twin takes in that asset’s data as it happens, keeps a memory of it, reasons about it, and — when you let it — acts on it.

The framework’s central idea is that every twin, regardless of domain, has the same internal structure. It is built from a fixed set of **layers**, stacked from raw data at the bottom to decision-making at the top. Each layer has a single responsibility and knows nothing about the layers above it. You choose which layers your asset needs and leave the rest out.

This matters because it means the framework contains no knowledge of pumps or turbines or fleets. It provides the structure — how data flows, how layers start and stop, how they talk to each other — and you provide the domain: what the sensors are called, what counts as a fault, what the asset should do about it.

The four layers

4 · AGENT	control, reasoning, and governance
3 · SERVICES	expose the twin to the outside world
2 · SIMULATION & MODEL	predict and forecast against a model
1 · DATA	ingest, store, clean, and score data
NETWORK	receive messages, validate their schema
PHYSICAL	publishers, simulators, protocol adapters

Read from the bottom up, each layer builds on the one below it:

1. **Data** is the foundation. It receives sensor readings (over MQTT, or any source you wire in), stores them in the right databases, smooths the noise out of them, condenses them into a single health score, and keeps a tamper-evident record of where each reading came from. Every layer above reads from here.
2. **Simulation and Model** runs a model of the asset alongside the real thing — a set of physics equations, a trained surrogate, or a forecaster. Comparing the model’s prediction to reality tells you when the asset is drifting away from how it should behave.

3. **Services** makes the twin visible. It mirrors the twin's current state into Eclipse Ditto (a standard digital-twin state store) and serves a REST and streaming API, so that a dashboard, a person, or a peer twin can ask the twin what it knows.
4. **Agent** is where the twin acts, reasons, and governs itself. These are one layer because they form a single ascent of autonomy, from a reflex to a deliberated decision. It has three tiers, and each gets a chapter of its own:
 - A **control tier** (chapter 07) is the deterministic floor. It checks readings against the thresholds you defined, drives a small state machine (running → warning → shutdown), and runs closed-loop PID control. It answers in milliseconds and gives the same answer to the same input every time, so a safety response never waits on an LLM.
 - A **reasoning tier** (chapter 08) holds a knowledge graph of the asset's components and failure modes, and runs LLM-powered agents that work out why a symptom is occurring, not merely that it is.
 - A **governance tier** (chapter 09) runs an OODA cycle — observe, orient, decide, act — choosing the next action from goals, a learned reinforcement-learning policy, or an LLM overseer, and escalating to a human when the stakes call for it.

In code these tiers are the `dyon.reactive`, `dyon.intelligent`, and `dyon.autonomous` modules. Keeping them separate is what lets you replace any one of them; keeping them under one layer is what lets the fast, certain path remain the safety net beneath the slow, clever one.

Beneath the Data layer sit two **enabling layers**, which you mostly configure rather than write. The Physical layer is the publishers, simulators, and protocol adapters that put readings on the wire. The Network layer receives those messages and validates their schema before the Data layer trusts them. Together they are the boundary between the physical asset and its digital counterpart. In the next chapter you will meet both: `dyon.infra` up starts the broker that is your Network layer, and a short simulator script stands in for the Physical one until a real asset is publishing.

Two further capabilities sit across the stack rather than inside it. **Connectors** let one twin reach another — query it, push to it, subscribe to it — so a twin can be useful alongside its peers rather than in isolation. **Collection twins** group many twins into one: a fleet, a hierarchy, or a network. Both have their own chapter.

You are never forced to use a layer. Because the layers are cumulative from the bottom, a twin can grow through three stages without being redesigned. A **connected** twin runs Data and Services, and monitors. A **predictive** twin adds Simulation and Model, and gains divergence detection. An **agentic** twin adds the Agent layer, and gains control, reasoning, and self-governance. Starting at the first stage costs you nothing at the third. You express which stage you are at in one place, which we will see next.

The four things you write

No matter how large the twin grows, you only ever author four kinds of thing. Everything else is supplied by the framework.

1 — A configuration object. `TwinConfig` names the asset, lists its sensors, and holds the addresses of your infrastructure. Every layer is handed this object and reads what it needs from it, so nothing about your asset is hardcoded anywhere else.

```
from dyon.core.config import TwinConfig, SensorFieldSpec
```

```

config = TwinConfig(
    asset_id="pump_001",
    asset_type="centrifugal_pump",
    sensor_fields=[
        SensorFieldSpec(name="temperature_c", nominal=45.0,
                        warn_threshold=60.0, crit_threshold=75.0),
        SensorFieldSpec(name="pressure_bar", nominal=4.2,
                        warn_threshold=3.5, crit_threshold=2.5,
                        threshold_direction="low"),
    ],
)

```

2 — A twin class. You subclass `AbstractDigitalTwin` and implement a single method, `build_layers()`. It returns a dictionary mapping a name to each layer instance you want active. This dictionary is your architecture choice — the layers you include are the layers your twin has.

```

from dyon.core.base import AbstractDigitalTwin

class PumpTwin(AbstractDigitalTwin):
    def build_layers(self):
        ts = InfluxAdapter(self.config)
        doc = MongoAdapter(self.config)
        cache = RedisAdapter(self.config)
        router = TelemetryRouter(self.config, self.bus,
                                ts_store=ts, doc_store=doc, cache=cache)

        return {
            "data": router,
            "reactive": ThresholdRuleEngine(self.config, self.bus,
                                           ts_store=ts, cache=cache,
                                           doc_store=doc),
        }

```

3 — Domain models, when a layer needs them. Some layers need asset-specific knowledge that the framework cannot guess: the differential equations for the Simulation and Model layer, the failure modes for the Agent layer's knowledge graph, the reward function for its governance tier. You provide these only when you add the layer that uses them. They are optional in exactly the same way the layers are.

4 — An entry point. A few lines that build the twin and hand it to the lifecycle manager, which runs it until you stop it.

```

import asyncio
from dyon.core.lifecycle import TwinLifecycle

if __name__ == "__main__":
    lifecycle = TwinLifecycle()
    lifecycle.add(PumpTwin(config))
    asyncio.run(lifecycle.run_forever())

```

How the layers run, and how they talk

When the lifecycle starts a twin, it calls `build_layers()` and then drives every layer through the same three phases. First it **initialises** each layer in the order you listed them, so a layer that prepares something

(the data stores, the Ditto thing) finishes before a later layer relies on it. Then it **starts** every layer at once — each layer runs its own loop concurrently, because once they are running they have no ordering between them. Finally, on shutdown, it **stops** them in reverse order, so a lower layer is never torn down while a higher one still depends on it.

Crucially, the layers never call each other's methods. A layer that needed to reach into another would have to know that other exists, and the whole point of the stack is that it does not. Instead, layers communicate through two indirect channels.

The first is **shared storage**. The data layer writes readings to InfluxDB; the control tier reads the latest reading back from InfluxDB. Neither holds a reference to the other. To add, remove, or replace a layer you change the `build_layers()` dictionary and nothing else.

The second is the **event bus**. When something worth announcing happens — a reading arrives, the state changes, an agent decides something — a layer publishes a `DomainEvent`. Any layer that cares subscribes to that event type and reacts. The publisher does not know or care who is listening.

```
# A layer announces something happened:
await self.bus.publish(DomainEvent(
    event_type="state.changed",
    source_layer="reactive",
    source_asset=self.config.asset_id,
    payload={"from": "running", "to": "warning"},
))

# A different layer listens for it:
self.bus.subscribe("state.changed", self._on_state_change)
```

A `DomainEvent` always carries its type, the layer and asset it came from, a free-form payload, a timestamp, and a severity (`info`, `warning`, or `critical`). Handlers run as independent tasks, and a handler that raises an exception is logged and isolated — one failing subscriber cannot bring down the publisher or the other subscribers.

Configuration is the single source of truth

Because every layer receives the same `TwinConfig`, that object is where all shared knowledge lives. The MQTT topic a layer subscribes to, the InfluxDB bucket it writes to, the list of sensor fields, the thresholds that define a fault — a layer reads each of these from `self.config`, never from a constant in its own code. Configuration is covered in full in chapter 03; for now just hold onto the principle, because it is what keeps the framework domain-agnostic. Change the config and you have changed the twin.

What the framework owns, and what you own

The framework owns	You own
How layers initialise, run, and stop	Your sensor field names and thresholds
How events flow between layers	The equations in your simulation model
How telemetry is routed to storage	Your failure modes and symptoms
The OODA loop and goal machinery	Your goal definitions and reward function
The LangChain agent wiring	Which LLM you connect to it

The reinforcement-learning scaffolding	What “good behaviour” means for the asset
The collection and connector plumbing	Which twins to group, and how they relate

The dividing line is consistent: the framework owns mechanism, you own meaning. With that frame in place, the next chapter builds a complete, running twin so you can see all of this in motion.

02 — Your First Twin

In this chapter you build a complete, running digital twin of a centrifugal pump and watch it work. It ingests sensor readings over MQTT, stores them, mirrors its state to Eclipse Ditto, and raises alarms when a reading crosses a threshold. In terms of chapter 01, you will use three of the four layers: the Data layer (with its MQTT ingest), the Services layer, and the control tier of the Agent layer. The Simulation and Model layer, and the Agent's reasoning and governance tiers, come in later chapters.

You will need Python 3.11+ and Docker with the Compose plugin. Install the framework with every capability included, since this guide will eventually use all of them:

```
pip install 'dyon[all]'
```

The bare `pip install dyon` is a deliberately lean core — MQTT, config, the REST layer, and the dashboard client — and each heavy capability lives in an optional extra (`stores` for the databases, `agents` for the LLM layers, `rl` for learning, and so on; the README tables them all). That matters when you deploy a twin and want only what it uses. While learning the framework, `[all]` saves you meeting those extras one pip command at a time — though if you do install narrowly, any feature whose extra is missing tells you the exact command that provides it.

Step 1 — Scaffold the project

The CLI writes a runnable starting point for you. From an empty directory:

```
dyon init --asset-type centrifugal_pump --name "Plant A Pump" --asset-id pump_001
```

This creates two files:

```
.env      — your asset's identity and the addresses of every backend
twin.py   — a twin class with a build_layers() method to fill in
```

The `.env` file is pre-filled with the framework's defaults, which match the infrastructure you are about to start, so you can leave it untouched for now.

Step 2 — Start the infrastructure

A twin needs backing services to talk to: an MQTT broker for incoming readings, databases for storage, and Eclipse Ditto for canonical state. The CLI generates a `docker-compose.yml` containing exactly the services your chosen layers need, and starts them:

```
dyon infra up --layers network,data,services,intelligent
```

The `--layers` flag is the important part. Each layer pulls in the containers it depends on, and nothing more:

Layer flag	Containers it adds
------------	--------------------

network	Mosquitto (MQTT broker)
data	InfluxDB, MongoDB, Redis, MinIO
services	The Eclipse Ditto stack (MongoDB, policies, things, gateway, nginx)
intelligent	Neo4j

The `network` layer also brings up the four data stores, because a twin that ingests over MQTT always needs somewhere to put what it ingests. Grafana is started in every case, on port 3000, for dashboards — the framework does not configure it, so you add InfluxDB as a data source through its own UI.

We include `intelligent` here so Neo4j is ready when you reach chapter 08; the pump twin in this chapter does not use it yet. Here is what each container is for and where to reach it:

Service	Port(s)	Role
Mosquitto	1883, 9001	MQTT broker (1883 plain, 9001 WebSocket)
InfluxDB	8086	Time-series sensor readings
MongoDB	27017	Events and asset metadata
Redis	6379	Latest-value cache and current state
MinIO	9000, 9002	Object storage (S3 API on 9000, console on 9002)
Eclipse Ditto	8080	Canonical twin state, behind an nginx proxy
Neo4j	7474, 7687	Knowledge graph
Grafana	3000	Dashboards

The command also records your layer choice in a `.dyon-layers` file, which the next step reads automatically.

Step 3 — Wait until everything is ready

Containers start in seconds, but services like InfluxDB and Ditto take up to a minute to finish initialising inside them. Check readiness before you run the twin:

```
dyon infra check
```

This pings every service that `infra up` started and prints a pass/fail line for each. Wait until all of them show `✓`.

Step 4 — Define the twin

Open `twin.py` and replace it with the pump twin below. It has three parts: the configuration that describes the asset, the twin class that selects the layers, and an entry point that runs it.

```
import asyncio
import logging
from dotenv import load_dotenv
from dyon.core.config import TwinConfig, SensorFieldSpec
from dyon.core.base import AbstractDigitalTwin
from dyon.core.lifecycle import TwinLifecycle
```

```

from dyon.data import InfluxAdapter, MongoAdapter, RedisAdapter
from dyon.data.writer import TelemetryRouter
from dyon.data.management import DataManagementPipeline
from dyon.network import MQTTIngestor
from dyon.services.ditto.client import DittoClient
from dyon.services.ditto.sync import DittoSyncService
from dyon.reactive import ThresholdRuleEngine

load_dotenv()
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(name)s %(levelname)s
%(message)s")
logging.getLogger("httpx").setLevel(logging.WARNING)

# 1. Describe the asset's sensors.
config = TwinConfig(
    sensor_fields=[
        SensorFieldSpec(name="temperature_c", nominal=45.0, noise_std=0.3,
            warn_threshold=60.0, crit_threshold=75.0,
            threshold_direction="high"),
        SensorFieldSpec(name="pressure_bar", nominal=4.2, noise_std=0.05,
            warn_threshold=3.5, crit_threshold=2.5,
            threshold_direction="low"),
        SensorFieldSpec(name="vibration_mm_s", nominal=1.5, noise_std=0.1,
            warn_threshold=2.5, crit_threshold=5.0,
            threshold_direction="high"),
    ]
)

# 2. Select the layers.
class PumpTwin(AbstractDigitalTwin):
    def build_layers(self):
        ts = InfluxAdapter(self.config) # time-series store
        doc = MongoAdapter(self.config) # event store
        cache = RedisAdapter(self.config) # latest-value cache
        ditto = DittoClient(self.config) # Eclipse Ditto client

        router = TelemetryRouter(self.config, self.bus,
            ts_store=ts, doc_store=doc, cache=cache)

        return {
            "network": MQTTIngestor(self.config, self.bus, router=router),
            "data": router,
            "data_mgmt": DataManagementPipeline(self.config, self.bus,
                ts_store=ts, cache=cache),
            "services": DittoSyncService(self.config, self.bus,
                ts_store=ts, cache=cache,
                doc_store=doc, ditto_client=ditto),
            "reactive": ThresholdRuleEngine(self.config, self.bus,
                ts_store=ts, cache=cache,
                doc_store=doc),
        }

# 3. Run it.
if __name__ == "__main__":
    lifecycle = TwinLifecycle()

```

```
lifecycle.add(PumpTwin(config))
asyncio.run(lifecycle.run_forever())
```

Notice how `build_layers()` mirrors chapter 01. You create the storage adapters once at the top, then hand them to whichever layers need them. The layers share data only through those adapters and the event bus, never by calling each other.

Each `SensorFieldSpec` carries everything the framework needs to know about one sensor: its `nominal` healthy value, how noisy it is, and the thresholds that define warning and critical conditions. The `threshold_direction` says which way is bad — temperature and vibration alarm when they rise above the threshold, pressure alarms when it falls below. Chapter 03 covers these fields in full.

Step 5 — Run

```
python twin.py
```

`TwinLifecycle` initialises the layers in order, starts their loops concurrently, and runs until you press `Ctrl+C`, at which point it stops every layer cleanly.

Here is what is now happening, second by second:

- On startup, the Ditto sync service creates a policy and a Thing for the pump, giving it `telemetry` and `health` features that external systems can read.
- The MQTT ingestor connects to Mosquitto and subscribes to `dt/pump_001/telemetry`. Any JSON message published there is picked up, stripped of non-numeric fields, and handed to the router.
- The router writes each reading to InfluxDB and updates the Redis cache, then announces a `telemetry.routed` event on the bus.
- Every 10 seconds, the data-management pipeline smooths the last few minutes of each field, computes its rate of change, flags its quality, and folds everything into a single 0–100 health score.
- Every 5 seconds, the Ditto sync service pushes the latest readings and health score into the Thing.
- Every 5 seconds, the rule engine reads the latest values and moves a state machine between `running`, `warning`, and `shutdown` (described below).

The twin is fully alive — but with no sensor publishing to it yet, it is watching an empty topic. Let us give it something to watch.

Step 6 — Drive it with the simulator

The framework ships a simulator that publishes synthetic readings to the same MQTT topic your twin listens on. In a second terminal, with `twin.py` still running, create and run this:

```
# simulate.py
import asyncio, logging
from dotenv import load_dotenv
from dyon.physical.simulator import GenericSimulator
from twin import config

load_dotenv()
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(name)s %(levelname)s")
```

```
%(message)s")

async def main():
    sim = GenericSimulator(config, publish_interval=1.0)
    await sim.run()

asyncio.run(main())
```

```
python simulate.py
```

The simulator reads the same `config`, so it knows each field's `nominal` value and noise. Every second it publishes a reading drawn from a Gaussian around nominal. Watch the twin's terminal: it routes each reading, and the health score settles near 100 because every value is in range.

Driving the state machine

The rule engine's state machine is the heart of this chapter, so it is worth seeing how it moves. On every evaluation it counts how many fields are in their warning zone and how many are in their critical zone, then:

- if any field is critical, it goes to `shutdown`;
- otherwise, if any field is in warning and it was `running`, it goes to `warning`;
- otherwise, if nothing is in warning and it was `warning`, it returns to `running`.

You can trigger each path by injecting a fault into the simulator. From a Python shell that imports your `running sim` object — or by editing `simulate.py` to call these — try a **warning-level** fault first:

```
sim.inject_fault({"temperature_c": 65.0}) # above warn (60), below crit (75)
# within ~5s the twin moves running → warning
sim.clear_fault()
# within ~5s it returns warning → running
```

A warning is self-clearing: once the value comes back into range, the engine moves the twin back to `running` on its own. A **critical** fault behaves differently, and the difference is deliberate:

```
sim.inject_fault({"temperature_c": 80.0}) # above crit (75)
# within ~5s the twin moves running → shutdown
sim.clear_fault()
# the twin stays in shutdown — it does not come back by itself
```

`shutdown` is a latching state. A pump that tripped on a critical fault should not silently restart the moment the reading looks better; bringing it back is a deliberate act. The engine exposes a `restart` transition for exactly this, and you decide when to fire it — from an operator action, a maintenance workflow, or the governance tier in chapter 09. Until then the twin holds at `shutdown`, which is the safe default.

Every state change is recorded as an event in MongoDB and published on the bus as a `state.changed` event, so any higher layer can react to it.

Step 7 — See where the data went

Everything the twin does leaves a trace you can inspect directly.

InfluxDB (<http://localhost:8086>, log in as `admin` / `password`) holds the time-series data in the `asset_telemetry` bucket, across three measurements:

Measurement	Written by	Contents
<code>asset_telemetry</code>	the router	raw readings as they arrive
<code>asset_processed</code>	the data-management loop	smoothed values, rate of change, quality
<code>asset_health</code>	the data-management loop	the composite 0–100 health score

Filter by the tag `asset_id = pump_001` to scope any query to this twin.

Redis (`localhost:6379`, no password) holds the fast-access state:

```
redis-cli
> GET dt:pump_001:state           # current FSM state
> GET dt:pump_001:latest:temperature_c  # latest cached value of a field
> KEYS dt:pump_001:*             # everything cached for this twin
```

MongoDB (`localhost:27017`, `admin` / `password`, database `digital_twin`) holds the document records. The `assets` collection has one metadata record per twin; the `events` collection has the discrete events — fault injections and state changes — newest entries last.

Eclipse Ditto (<http://localhost:8080>, basic auth `ditto` / `ditto`) holds the canonical state, which is what an external dashboard or integration would read:

```
curl -u ditto:ditto http://localhost:8080/api/2/things/org.example:pump_001
curl -u ditto:ditto http://localhost:8080/api/2/things/org.example:pump_001/
features/telemetry/properties
curl -u ditto:ditto http://localhost:8080/api/2/things/org.example:pump_001/
features/health/properties
```

The `health` feature carries both the `health_score` and the current `operational_state`, so a consumer can read the twin's condition in one request.

You now have a working twin: it ingests, stores, processes, mirrors, and reacts. The remaining chapters add capability one layer at a time, but the shape never changes — you describe the asset in `TwinConfig`, you select layers in `build_layers()`, and you run the lifecycle. Next, chapter 03 looks closely at that configuration object, because getting it right is what makes everything above it work.

03 — Configuration and Sensors

Chapter 01 called `TwinConfig` the single source of truth, and chapter 02 used it without dwelling on it. Now we look at it properly, because almost everything the framework does is driven by what you put here. Get the configuration right and the layers above it largely wire themselves; get it wrong and the symptoms show up far from the cause.

`TwinConfig` does two jobs. It describes the asset — its identity and its sensors — and it holds the addresses and credentials of your infrastructure. The first is unique to your twin and lives in code. The second changes between machines and lives in environment variables. We will take them in that order.

Describing the asset

```
from dyon.core.config import TwinConfig, SensorFieldSpec

config = TwinConfig(
    asset_id="pump_001",
    asset_type="centrifugal_pump",
    asset_name="Plant A Pump",
    sensor_fields=[ ... ],
)
```

The three identity fields are short but load-bearing. `asset_id` in particular is woven through everything: it becomes part of the MQTT topics the twin listens on, the keys it writes to Redis, the tag it attaches to every InfluxDB point, and the ID of its Ditto Thing. Two twins with different `asset_id`s can therefore share the same infrastructure without their data ever mixing. `asset_type` and `asset_name` are descriptive — they appear in metadata and logs.

`sensor_fields` is the heart of the configuration: the list of channels your asset measures. Everything above the configuration reads this list to know what to store, what to watch, and what to model.

The anatomy of a sensor field

Each `SensorFieldSpec` describes one channel.

```
SensorFieldSpec(
    name="temperature_c",      # the field's name, used everywhere
    unit="°C",                 # for display only
    nominal=45.0,               # the expected healthy value
    noise_std=0.3,              # how much it naturally fluctuates
    warn_threshold=60.0,        # the edge of the warning zone
    crit_threshold=75.0,        # the edge of the critical zone
    threshold_direction="high", # which direction is dangerous
)
```

`name` is the only required field; everything else has a default. The name is the exact string the rest of the framework uses — the key it expects in an incoming MQTT message, the field it writes to InfluxDB, the label it shows in Ditto. Choose it carefully and use it consistently.

`nominal` and `noise_std` describe healthy behaviour. The simulator from chapter 02 uses them to generate realistic readings (a Gaussian centred on `nominal` with standard deviation `noise_std`). A field with `nominal` left as `None` is treated as a derived or computed channel — the simulator skips it, since there is no natural value to generate.

`warn_threshold`, `crit_threshold`, and `threshold_direction` define what counts as a fault. The direction tells the framework which way is bad:

- "high" means the field alarms when it rises above the thresholds — natural for temperature, vibration, or pressure in a system that must not over-pressurise.
- "low" means it alarms when it falls below them — natural for a pressure, flow rate, or fluid level that must not drop.

So a high-direction field with `warn=60`, `crit=75` warns above 60 and goes critical above 75, while a low-direction field with `warn=3.5`, `crit=2.5` warns below 3.5 and goes critical below 2.5.

Sensor	Direction	Warn	Crit	Reads as
<code>temperature_c</code>	high	60	75	warn above 60, critical above 75
<code>pressure_bar</code>	low	3.5	2.5	warn below 3.5, critical below 2.5
<code>vibration_mm_s</code>	high	2.5	5.0	warn above 2.5, critical above 5.0
<code>flow_rate_lpm</code>	low	80	50	warn below 80, critical below 50

Thresholds are optional. A channel you only feed into a model — a `speed_rpm` you never alarm on — simply leaves both thresholds unset, and the control tier ignores it:

```
SensorFieldSpec(name="speed_rpm", nominal=1450.0, noise_std=5.0)
```

A field is only watched for faults when it has both a warn and a crit threshold.

What the configuration computes for you

Once `sensor_fields` is set, `TwinConfig` exposes a handful of derived properties so the layers never have to reassemble this information themselves. These are the exact values the layers read at runtime:

```
config.field_names
# ['temperature_c', 'pressure_bar', 'vibration_mm_s', 'speed_rpm']

config.thresholds
# {'temperature_c': {'warn': 60.0, 'crit': 75.0, 'low': False},
#  'pressure_bar': {'warn': 3.5, 'crit': 2.5, 'low': True},
#  'vibration_mm_s': {'warn': 2.5, 'crit': 5.0, 'low': False}}
# speed_rpm is absent – it has no thresholds.

config.topic_telemetry # 'dt/pump_001/telemetry'
config.topic_control   # 'dt/pump_001/control'
config.topic_state     # 'dt/pump_001/state'
config.thing_id        # 'org.example:pump_001'
```

`thresholds` is the one the control tier and the health calculator depend on: each entry collapses the warn value, the crit value, and the direction (as the boolean `low`) into the form those layers consume.

Because they read it from here, threshold values exist in exactly one place — the `SensorFieldSpec` you wrote — and nowhere else.

Connecting to infrastructure

The rest of `TwinConfig` describes where your backends live. These values rarely belong in code, because they differ between your laptop, a staging box, and production. Instead the framework reads them from environment variables, which you keep in a `.env` file that `TwinConfig` loads automatically.

The naming rule has two parts. Every variable starts with the prefix `DT_`. For the flat identity fields, that prefix and the field name are all you need:

```
DT_ASSET_ID=pump_001
DT_ASSET_TYPE=centrifugal_pump
DT_API_PORT=8501
```

The infrastructure settings are grouped into sub-sections — `mqtt`, `influx`, `mongo`, and so on — and you reach into a section with a double underscore between the section and the field:

```
DT_MQTT__BROKER=192.168.1.50
DT_MQTT__PORT=1883

DT_INFLUX__URL=http://influx.internal:8086
DT_INFLUX__TOKEN=my-production-token
DT_INFLUX__ORG=production

DT_LLM__PROVIDER=anthropic
DT_LLM__MODEL=claude-sonnet-4-6
DT_LLM__API_KEY=sk-ant-...
```

The double underscore is what tells the loader you mean the `broker` field inside the `mqtt` section. A single underscore would be read as a flat top-level name and ignored, so this is the one detail to be careful about.

`sensor_fields` is the only part of the configuration that cannot come from the environment, because it is a list of structured objects rather than a string. It always lives in your Python code.

Dev mode and production mode

One sub-section deserves its own introduction, because it governs how seriously the twin treats all the others. The `security` section has a `mode` field with two settings, and the default is `dev`:

```
DT_SECURITY__MODE=dev # the default – zero-config local development
```

In dev mode the twin starts with whatever configuration it has, including the factory defaults for every credential above. That is deliberate: on your own machine, against infrastructure that `dyon infra up` just created, being able to run `python twin.py` with an empty `.env` is worth more than credential discipline. Dev mode is also why the API binds to `127.0.0.1` unless you say otherwise — a twin you are developing is reachable from your browser and from nothing else.

The moment a twin is exposed beyond your machine, switch the mode:

```
DT_SECURITY__MODE=production
DT_SECURITY__API_KEY=<a long random string>
DT_SECURITY__CORS_ORIGINS='["https://ops.example.com"]'
```

In production mode the twin refuses to start until its configuration is actually secure: every factory-default credential must be replaced, an API key must be set, and CORS must name explicit origins. The error message lists each offending setting, so fixing a refusal is a matter of working down the list. The API key, once set, is required on every `/api/*` route — chapter 06 shows how clients present it.

This is a fail-fast design. A production twin with a default MongoDB password is not a twin with a warning in its logs; it is a twin that never comes up. The check runs once, at startup, and costs nothing afterwards.

Running several twins side by side

Because identity flows from `asset_id`, running more than one twin on the same machine is just a matter of giving each a different one. Inline environment variables override the `.env` file, so you can launch two pumps from the same `twin.py`:

```
# terminal 1
DT_ASSET_ID=pump_001 DT_API_PORT=8501 python twin.py

# terminal 2
DT_ASSET_ID=pump_002 DT_API_PORT=8502 python twin.py
```

Or set the values directly in code, which is the pattern collection twins use in chapter 10:

```
config_001 = TwinConfig(asset_id="pump_001", api_port=8501, sensor_fields=...)
config_002 = TwinConfig(asset_id="pump_002", api_port=8502, sensor_fields=...)
```

Either way, each twin keeps its own topics, its own cache keys, and its own Ditto Thing.

The sub-config reference

You will rarely set these by hand — the defaults match the infrastructure `dyon infra up` generates — but this is what each section controls and the default it falls back to:

Section	Key fields	Default
<code>mqtt</code>	broker, port, keepalive, username, password	localhost:1883
<code>influx</code>	url, token, org, bucket	localhost:8086 / digital_twin / asset_telemetry
<code>mongo</code>	uri, db	localhost:27017 / digital_twin
<code>redis</code>	url, db	redis://localhost:6379 / db 0
<code>minio</code>	endpoint, access_key, secret_key, bucket	localhost:9000 / digital-twin-assets
<code>ditto</code>	url, user, password, namespace	localhost:8080 / org.example
<code>neo4j</code>	uri, user, password	bolt://localhost:7687

llm	provider, model, api_key, base_url, temperature, timeout_s, max_tokens, max_retries	openai / gpt-4o-mini
security	mode, api_key, cors_origins	dev mode, no key

Two sections carry fields worth knowing before you need them. `mqtt` includes `tls`, `tls_ca_certs`, and `tls_insecure`, so a broker outside your machine can be reached over an encrypted connection (typically on port 8883). And `llm` bounds every call the diagnostic agent makes: `timeout_s` caps how long a single request may take, `max_tokens` caps how long a reply may grow, and `max_retries` covers transient provider errors. The defaults are sensible, and chapter 08 explains what they protect you from. With the configuration understood, we can descend to the bottom of the stack. The next chapter is the Data layer — where every reading the twin receives is stored, cleaned, and turned into the health score the rest of the framework relies on.

04 — The Data Layer

Everything a twin knows about its asset starts here. The data layer answers one question — what is the asset doing right now? — and it answers it durably, so the knowledge survives a restart, and quickly, so the layers above it can ask many times a second. Those upper layers never touch sensor hardware or the network themselves; they read what the data layer has already collected and prepared.

The layer has two halves. The first takes readings in and stores them. The second runs on a timer and turns those raw readings into something more useful — smoothed values, trends, and a single health score. We will build up both, then look at three extensions for twins whose “data” is not a stream of numbers: free text, stateful sessions, and an auditable trail of decisions.

Why four stores instead of one

A twin keeps four kinds of data, and no single database is good at all four. Each backend is chosen for the access pattern it serves.

InfluxDB holds time series: the same quantity measured over and over (temperature at 10:00:01, at 10:00:02, and so on). It is built to answer “give me every temperature reading from the last three hours” almost instantly, which is exactly the question the twin’s models and reasoning agents ask. A general-purpose database can store the same numbers but is far slower at this particular shape of query.

MongoDB holds events: discrete things that happened, rather than continuous measurements. A state change from `running` to `warning` is an event; a fault injection is an event; an agent’s decision is an event. These records have irregular shapes, so a document store that accepts arbitrary JSON suits them better than a fixed table.

Redis holds the latest value of everything, in memory. Reading the current temperature from Redis takes well under a millisecond; reading it from InfluxDB takes tens of milliseconds. Layers that need the current value on a tight loop — the rule engine firing every five seconds, a dashboard polling — read from Redis. Layers that need history read from InfluxDB.

MinIO holds binary objects: trained model files, firmware images, large documents. You do not query these continuously; you load one at startup and keep it in memory.

You create an adapter for each store you need at the top of `build_layers()`, then pass them to the layers that use them:

```
def build_layers(self):
    ts      = InfluxAdapter(self.config)    # time series
    doc     = MongoAdapter(self.config)    # events and metadata
    cache   = RedisAdapter(self.config)    # latest values and current state
    obj     = MinIOAdapter(self.config)    # binary objects (only if needed)
    ...
```

The TelemetryRouter: the way in

The router is the data layer's entry point. It receives a validated reading — a plain dictionary of field names to numbers — and fans it out to the stores that need it. “Fan out” is the key idea: one reading arrives once and is forwarded to several destinations in a single step.

```
router = TelemetryRouter(self.config, self.bus,
                        ts_store=ts, doc_store=doc, cache=cache)
```

For each reading, the router keeps only the fields that match your `sensor_fields` and discards the rest. It writes those numbers to InfluxDB under the `asset_telemetry` measurement, updates the Redis cache with each field's latest value, and records a `last_seen` timestamp. It then publishes a `telemetry.routed` event so any interested layer learns that fresh data has landed. MongoDB is not part of this hot path — the router writes there only when a reading is flagged as a fault injection, because normal telemetry is a time series, not an event.

When your twin ingests over MQTT, the `MQTTIngestor` calls `router.route()` for you, so you never invoke it directly. If your data arrives some other way — a REST endpoint, a serial port, a database poll — your code calls `await router.route(reading)` itself. Either way, the router is the single doorway through which readings enter the twin.

The DataManagementPipeline: from raw to useful

Raw readings are noisy and hard to reason about. The pipeline runs on a timer (ten seconds by default) and, for each field, looks back over the last few minutes of data to produce three derived signals:

- `{field}_smooth` — a rolling average that takes the noise out of the reading;
- `{field}_roc` — the rate of change, the difference between the two most recent values, which tells you the direction the field is moving;
- `{field}_quality` — a simple flag: `1.0` when the field is in range, `2.0` in the warning zone, `3.0` in the critical zone.

```
DataManagementPipeline(self.config, self.bus,
                       ts_store=ts, cache=cache,
                       interval=10,          # run every N seconds
                       smooth_window=5,      # average over the last N samples
                       lookback_minutes=5) # how far back to read
```

These derived signals are written to InfluxDB under the `asset_processed` measurement and cached in Redis.

The pipeline then computes the **health score**, which is the data layer's most important output. It compresses every threshold check into one number from 0 to 100. Each field with thresholds carries equal weight; a field sitting in its warning zone costs half of its weight, and one in its critical zone costs all of it. A twin with every field in range scores 100, and one with every field critical scores 0. The score is written to the `asset_health` measurement and cached.

The value of this compression is that a higher layer can make a decision from a single number. The governance tier's goal planner, in chapter 09, treats `health_score < 50` as a risk signal without ever needing to know which field dropped — the health score has already done that aggregation.

How the layers above read data

Higher layers do not receive readings from the router. They read back from InfluxDB and Redis using the adapters they were given:

```
# Latest value of one field, from InfluxDB:
val = ts.get_latest("temperature_c")

# Latest value, from the Redis cache (faster):
val = cache.get_latest_cached("temperature_c")

# The current FSM state:
state = cache.get_state()

# Recent events, newest first:
events = doc.get_recent_events(n=20)

# Recent events of one type (used by the reasoning tier):
decisions = doc.get_events_by_type("mas_agent_AnomalyDetectionAgent", n=50)

# Several fields over a window, in one query – returns
# {field: [{"ts": unix_seconds, "value": float}, ...], ...}:
history = ts.query_recent_fields(
    fields=["temperature_c", "pressure_bar", "vibration_mm_s"], minutes=120)
```

The last call is how a dashboard pre-loads a chart: a single InfluxDB request for many fields over a window.

What ends up in InfluxDB

By the time several layers are running, InfluxDB holds a handful of measurements, each written by a different part of the twin. Every point is tagged with `asset_id`, so many twins can share one InfluxDB instance without their data mixing.

Measurement	Written by	Contents
<code>asset_telemetry</code>	TelemetryRouter	raw sensor readings
<code>asset_telemetry</code>	TextIngestor	numeric signals derived from text
<code>asset_processed</code>	DataManagementPipeline	smoothed values, rate of change, quality flags
<code>asset_health</code>	DataManagementPipeline	the composite health score
<code>asset_simulation_{model_name}</code>	ModelRunner (ch. 05)	model predictions
<code>asset_residuals_{model_name}</code>	ModelRunner (ch. 05)	measured minus predicted, per field

When the data is text: TextIngestor

Not every twin measures numbers. Some receive language — operator notes, maintenance logs, inspection reports, support tickets. `TextIngestor` lets the rest of the framework treat that text as if it were ordinary telemetry, by deriving a few numeric signals from each piece of text and writing them to InfluxDB, while keeping the original text in MongoDB for audit and search.


```

from dyon.data.text_ingestor import TextIngestor

ingestor = TextIngestor(config, self.bus, ts_store=ts, doc_store=doc,
                        event_type="operator_note")

signals = await ingestor.ingest(
    session_id="sess_001",
    source_label="operator",
    content="The pump is making a grinding noise.",
)
# signals == {"sentiment_score": ..., "text_length": ..., "is_question": ...}

```

Each ingested message yields three signals: a `sentiment_score` from 0 (very negative) to 1 (very positive), the `text_length` in characters, and `is_question`, which is `1.0` when the text ends in a question mark. Sentiment is scored with VADER by default, but you can pass your own `sentiment_fn` — a callable from text to a float — to swap in a fine-tuned model:

```

ingestor = TextIngestor(..., sentiment_fn=my_model.score)

```

The signals land in `asset_telemetry`, tagged with the `session_id`, so they flow through smoothing, health scoring, and the control tier exactly like sensor data. This is the bridge that lets a twin whose data arrives as language run on exactly the same machinery as one whose data arrives as numbers.

When interactions have a beginning and end: sessions

Some assets are not continuous — they have bounded episodes. A production run, a maintenance job, a clinical consultation: each starts, runs through phases, and ends. `SessionContext` holds the live state of one such episode, and `SessionStore` persists it (in Redis, by default).

```

from dyon.session.context import SessionContext, SessionStore

store: SessionStore[SessionContext] = SessionStore(cache, ttl=3600)
ctx = store.new_session(primary_entity_id="device_42")
ctx.add_event("note", "started maintenance")
store.save(ctx)

reloaded = store.load(ctx.session_id)

```

A `SessionContext` is deliberately domain-neutral. It carries a generated `session_id`, a `primary_entity_id` and `secondary_entity_id` for the entities the session is about, a `phase` string, a `health_score`, an `alert_count`, an `event_history` you append to with `add_event()`, an `outcome` once it ends, and an `extra` dictionary for anything else. It also tracks `started_at` and `last_updated`, and exposes `elapsed_seconds`.

When your domain needs more than these generic fields, you do not edit `SessionContext` — you subclass it in your own code and tell the store to use your subclass:

```

from dataclasses import dataclass, field
from dyon.session.context import SessionContext, SessionStore

```

```
@dataclass
class JobSessionContext(SessionContext):
    technician_id: str = ""
    job_type: str = ""
    fault_codes: list = field(default_factory=list)

def make_job_store(cache) -> SessionStore[JobSessionContext]:
    return SessionStore(cache, context_class=JobSessionContext)
```

The store is generic over the context type, so `load()` and `new_session()` return your subclass directly, with no casting.

When decisions must be auditable: ProvenanceLog

For twins that take consequential actions — offering a discount, escalating to a human, applying a control change — you often need a record that cannot be quietly rewritten. `ProvenanceLog` provides one: an append-only, hash-chained log stored in a capped MongoDB collection.

```
from dyon.data.storage.provenance import ProvenanceLog
from pymongo import MongoClient

provenance = ProvenanceLog(MongoClient(config.mongo.uri), db_name=config.mongo.db)

provenance.append(
    actor="autonomous_controller",
    inputs={"session_id": "sess_001", "discount": 0.10},
    output_summary="discount_offered",
    session_id="sess_001",
)
```

Each entry stores a SHA-256 hash of its inputs, the previous entry's hash, and its own hash computed over the previous hash plus its body. Because every record is chained to the one before it, altering any past record changes its hash and breaks the link at the next record. If you replicate the most recent hash somewhere outside the database, `provenance.verify_chain()` can then confirm the entire history is intact:

```
ok = provenance.verify_chain() # True if every link checks out
```

You can also retrieve a session's records with `query_by_session()` or the latest entries with `query_recent()`.

When you need SQL: PostgresAdapter

InfluxDB and MongoDB cover time series and documents, but some data is genuinely relational — versioned rule tables, compliance records, anything you want to JOIN. `PostgresAdapter` is a thin async wrapper over a connection pool for those cases.

```
from dyon.data.storage.postgres import PostgresAdapter
```

```
pg = PostgresAdapter("postgresql://dt_user:secret@localhost:5432/dyon")
await pg.connect() # once, at startup

await pg.execute("INSERT INTO rule_versions (name, body) VALUES ($1, $2)",
                 "high_temp_rule", '{"threshold": 70}')
rows = await pg.fetch("SELECT * FROM rule_versions WHERE name = $1",
                      "high_temp_rule")
row = await pg.fetchrow("SELECT * FROM rule_versions WHERE id = $1", 42)
count = await pg.fetchval("SELECT COUNT(*) FROM rule_versions")

await pg.close()
```

Postgres sits slightly apart from the other backends in two ways. It is not provisioned by `dyon infra up`, so you run the database yourself; and it has no sub-section in `TwinConfig`, so you pass a DSN string straight to the adapter rather than reading it from a `DT_POSTGRES__*` variable. Its main use inside the framework is the control tier's optional `RuleRepository`, which we meet in chapter 07.

Replacing a backend

Every adapter is defined by a small protocol, so you can substitute your own. To use TimescaleDB instead of InfluxDB, write a class with the same handful of methods the framework calls — `write_point`, `query_recent`, `get_latest`, `query_recent_fields`, and `close` — and pass it wherever `InfluxAdapter` would go. Nothing else in the twin changes, because nothing else knows which time-series store it is talking to. If you replace the document store, implement `get_events_by_type` as well, since the reasoning tier relies on it.

With the foundation in place — readings stored, cleaned, scored, and auditable — the twin knows what its asset is doing. The next chapter adds a model of what it should be doing, so the two can be compared.

05 — The Simulation and Model Layer

The data layer tells you what the asset is doing. This layer adds a model of what it should be doing, and watches the gap between the two. That gap — the difference between a prediction and the matching real reading — is called the **residual**, and it is the most sensitive fault detector the framework has.

A threshold alarm only fires once a reading crosses a hard limit, which catches sudden failures but misses slow decline. A residual catches the decline. If the model says the temperature should be 46 °C and the sensor reads 52 °C, the residual is +6 °C even though 52 is far below any alarm line. That tells you the asset is running six degrees hotter than physics allows for at this operating point — a discrepancy that often appears hours or days before any threshold is breached.

The catch is that residuals are only as trustworthy as the model. A poorly calibrated model produces large residuals all the time, drowning the real signal in noise, so you validate a model against known-healthy operation before you rely on it. With that caveat understood, let us build one.

Three kinds of model

The framework supports three ways to predict an asset's behaviour, and they suit different situations.

A **physics model** encodes the actual governing equations — fluid dynamics for a pump, the heat equation for a furnace wall. You write it from domain knowledge. It is interpretable, because you can read the equations and reason about its predictions, but it takes expertise to write and its numerical integration can be slow.

A **surrogate model** is a machine-learning model trained to approximate a physics model. You run the physics model many times across the full range of inputs, record the results, and fit a neural network or regressor to those input-output pairs. The surrogate runs far faster, which makes it suitable for tight control loops, at the cost of interpretability and of being valid only within the range it was trained on.

A **forecaster** ignores physics entirely and projects a single field forward from its own history, finding daily and weekly cycles and extrapolating them. It answers “where will this value be in 24 hours if conditions hold?” rather than “what should this value be right now?”

The first two plug into this layer's runner and feed residuals. The forecaster is a standalone tool you call on demand, which we cover at the end.

The ModelRunner

Physics and surrogate models share one interface — the `TwinModel` protocol — so the runner can drive any mix of them the same way. You register your models with a `ModelRunner` and it steps them on a loop:

```
from dyon.simulation import ModelRunner

ModelRunner(self.config, self.bus,
             ts_store=ts,
             models=[physics_model, surrogate_model], # any mix
             step_interval=1.0,                       # step every second
```

```

        residual_anomaly_threshold=10.0)                # alert when |residual|
exceeds this

```

On each interval, for every model, the runner reads the latest value of each sensor field from InfluxDB to use as inputs, calls the model's `step()`, and writes the prediction to `asset_simulation_{model_name}`. It then pairs each predicted field with the real reading, writes the differences to `asset_residuals_{model_name}`, and — if any residual exceeds the threshold — publishes a `simulation.anomaly_detected` event so the Agent layer can respond.

The pairing relies on a naming convention: a model returns each prediction under the key `sim_{field}`, and the runner strips the `sim_` prefix to find the real field it should compare against. Get the field names right and residuals line up automatically; this is why a model's output names must match your `SensorFieldSpec` names exactly.

Writing a physics model

You subclass `ODEModel` and implement one method, `derivatives()`, which returns the rate of change of each state variable. Its signature is `(self, t, y, u)`: `t` is time in seconds, `y` is the state vector as a NumPy array, and `u` is the single control input (a speed or power setpoint).

```

import numpy as np
from dyon.simulation import ODEModel

class PumpODEModel(ODEModel):
    model_name = "pump_physics"  # becomes the InfluxDB measurement label

    def derivatives(self, t, y, u):
        T, P, Q = y               # temperature, pressure, flow
        n = u / 1450.0           # speed normalised to nominal

        Q_heat = (1.0 - 0.75) * abs(P * Q) / 600.0  # heat from inefficiency
        dT = (Q_heat - (T - 25.0)) / 120.0         # heat in, decay toward 25
        °C ambient

        dP = (4.2 * n**2 - P) / 5.0                 # pressure tracks speed²
        dQ = (120.0 * n - Q) / 3.0                  # flow tracks speed

        return [dT, dP, dQ]

model = PumpODEModel(
    initial_state=np.array([25.0, 4.2, 120.0]),
    state_names=["temperature_c", "pressure_bar", "flow_rate_lpm"],
    control_field="speed_rpm",  # which input field drives the model
    nominal_input=1450.0,      # value to use if that field is missing
)

```

`ODEModel` integrates your equations with SciPy's `solve_ivp` each step and returns the new state as `sim_temperature_c`, `sim_pressure_bar`, and so on. The `state_names` you give must match your sensor field names, so the runner can pair each `sim_` output with its real counterpart.

Writing a surrogate model

If you have a trained network exported to ONNX, wrap it with `ONNXSurrogate` :

```
from dyon.simulation import ONNXSurrogate

surrogate = ONNXSurrogate(
    model_path="models/pump_surrogate.onnx",
    input_fields=["speed_rpm", "pressure_bar"],      # what the model reads
    output_fields=["temperature_c", "flow_rate_lpm"], # what it predicts
)
```

For a fitted scikit-learn estimator, use `SKLearnSurrogate` the same way:

```
import joblib
from dyon.simulation import SKLearnSurrogate

surrogate = SKLearnSurrogate(
    model=joblib.load("models/pump_regressor.joblib"),
    input_fields=["speed_rpm", "pressure_bar"],
    output_fields=["temperature_c"],
)
```

Both are stateless: they read the named input fields, run inference, and return predictions as `sim_<output_field>`. As with the physics model, the output names must match your sensor fields for residuals to align.

Writing any other model

`TwinModel` is just two attributes and two methods, so anything that can predict from inputs can be a model — a lookup table, a rule of thumb, a remote service:

```
from dyon.core.types import ModelType

class LookupTableModel:
    model_name = "lookup_table"
    model_type = ModelType.ML

    def __init__(self, table):
        self._table = table

    def step(self, dt, inputs):
        speed = inputs.get("speed_rpm", 1450.0)
        nearest = min(self._table, key=lambda k: abs(k - speed))
        return {"sim_temperature_c": self._table[nearest]["temp"]}

    def reset(self):
        pass
```

The `model_type` is a label from the `ModelType` enum in `dyon.core.types` (such as `PHYSICS`, `SURROGATE`, `DISCRETE_EVENT`, or `ML`); it is descriptive metadata and does not change how the runner treats the model.

For processes that are better described by events than by differential equations — queues, batch runs, maintenance cycles — `SimPyModel` wraps a SimPy process as a `TwinModel` :

```
from dyon.simulation import SimPyModel

def machine_process(env, state):
    while True:
        state["throughput"] = state.get("throughput", 100.0)
        yield env.timeout(1)

model = SimPyModel(process_fn=machine_process,
                    output_fields=["throughput"],
                    initial_state={"throughput": 100.0})
```

Each `step(dt, inputs)` advances the SimPy clock by `dt` and returns the current state as `sim_<field>`.

Running several models at once

Because each model writes to its own measurement, you can run several together and compare them — a physics model against a surrogate, or two surrogates trained differently:

```
ModelRunner(self.config, self.bus, ts_store=ts,
            models=[physics_model, onnx_surrogate, sklearn_model])
# writes asset_simulation_pump_physics, asset_simulation_onnx_surrogate, ...
# plus the matching residual measurements
```

Side by side in Grafana, their residual streams show you which model tracks the real asset most faithfully.

The forecaster

`ProphetForecaster` is not a `TwinModel` and is not driven by the runner. It is a utility you call when you want to look ahead: it fits a Prophet model to a field's recent history and projects it forward.

```
from dyon.simulation import ProphetForecaster

fc = ProphetForecaster(ts_store=ts, field="temperature_c")
fc.fit(lookback_hours=48)          # train on the last two days
forecast = fc.predict(periods=24)  # project 24 hours ahead
# [{"ds": datetime, "yhat": float, "yhat_lower": float, "yhat_upper": float}, ...]
```

The reasoning and governance tiers use this when a decision depends on where a field is heading, not just where it is now.

Where the data comes from

It is worth being clear about two things the framework calls “simulation”, because they live in different packages and do different jobs.

The models in this chapter live in `dyon.simulation`. They run inside the twin, alongside whatever data is arriving, and their only job is to predict and produce residuals. They are never a source of sensor data.

The `GenericSimulator` from chapter 02 lives in `dyon.physical`. It runs as a separate process and publishes synthetic readings to the twin's MQTT topic, standing in for hardware you do not have yet. The twin cannot tell its readings apart from a real sensor's.

The two are independent. `ModelRunner` predicts against whatever readings flow in, whether those come from `GenericSimulator` or from real hardware. Moving from one to the other changes nothing in the twin: you stop the simulator and let your real sensors publish to the same MQTT topic. If your hardware does not speak MQTT, a thin publisher bridges it — `SerialPublisher` for serial devices, or any code that calls `router.route(readings)` directly.

```
from dyon.physical.simulator import SerialPublisher

pub = SerialPublisher(config, port="/dev/ttyUSB0", baud=9600, parser=my_parser)
pub.connect()
```

With a model running and residuals flowing, the twin can now tell not only that a reading is high, but that it is higher than it should be. The next chapter makes all of this visible to the outside world.

06 — The Services Layer

Every layer so far has run entirely inside the twin process. Readings go into databases, layers talk over the event bus, and none of it is visible from outside. The services layer opens the twin up. It gives the twin a public face that dashboards, other software, and other twins can connect to, through two components: a synchronised state record in Eclipse Ditto, and a FastAPI web API.

Eclipse Ditto: a standard place to read twin state

Eclipse Ditto is a separate program — it runs in its own container — that keeps a structured, always-current record for each twin, readable by any HTTP client. Think of it as one live JSON document per twin that always reflects the latest state. The framework keeps that document up to date for you.

The point of routing state through Ditto rather than exposing your own databases is standardisation. Every Dyon twin presents the same Ditto interface, whatever its domain. A collection twin that watches twenty component twins (chapter 10) does not need to know each one's internal storage — it reads them all the same way. The interface stays stable while the internals are free to change.

Every twin's Ditto record, called a **Thing**, has two features:

Feature	Holds
telemetry	the latest value of each sensor field
health	the <code>health_score</code> and the <code>operational_state</code> (running/warning/shutdown)

You add Ditto to a twin with the client and the sync service, exactly as the pump twin did in chapter 02:

```
from dyon.services.ditto.client import DittoClient
from dyon.services.ditto.sync import DittoSyncService

ditto = DittoClient(self.config)

# inside build_layers():
"services": DittoSyncService(self.config, self.bus,
                             ts_store=ts, cache=cache,
                             doc_store=doc,          # needed for the events
endpoint
                             ditto_client=ditto,
                             sync_interval=5),       # push every N seconds
```

When the layer initialises, it waits for the Ditto gateway to come up, then creates a policy and the Thing (with both features). Because it creates them with an idempotent PUT, restarting the twin simply re-establishes the same Thing. Every `sync_interval` seconds afterwards, it writes the latest readings into the `telemetry` feature and the current health and state into the `health` feature.

Any system can then read the twin with a single request:

```
curl -u ditto:ditto http://localhost:8080/api/2/things/org.example:pump_001
curl -u ditto:ditto http://localhost:8080/api/2/things/org.example:pump_001/
```

```
features/telemetry/properties
curl -u ditto:ditto http://localhost:8080/api/2/things/org.example:pump_001/
features/health/properties
```

This is exactly how collection twins query their members in chapter 10.

If you would rather not run Ditto, leave the sync service out of `build_layers()` and drop the `ditto` container from your compose file. Neither the Data layer nor the Agent layer's control tier depends on it; only the API endpoints that read from Ditto stop working, and the rest of the API keeps serving.

The FastAPI web API

The framework builds the twin's web API for you through a `create_app()` factory. You give it the config and a `ServiceRegistry` — a small directory that lets the API routes find the running layers they need — and it returns a FastAPI app you serve alongside the twin:

```
import asyncio, uvicorn
from dyon.services.api import create_app
from dyon.services.base import ServiceRegistry
from twin import PumpTwin, config

async def main():
    twin = PumpTwin(config)
    await twin.initialise()

    registry = ServiceRegistry()
    registry.register(twin.layers["services"]) # the DittoSyncService

    app = create_app(config, registry)
    server = uvicorn.Server(uvicorn.Config(
        app, host=config.api_host, port=config.api_port, log_level="warning"))
    await asyncio.gather(twin.start(), server.serve())

asyncio.run(main())
```

The registry is the link between the web layer and the twin's internals. Each service registers under a known name, and the API routes look services up by that name. The built-in routes expect three names, and each route only works when the matching service is registered:

Method	Path	Needs registered	Returns
GET	/health	—	liveness + the twin's error counters
GET	/api/twin/state	ditto_sync	the full Ditto Thing
GET	/api/twin/telemetry	ditto_sync	the telemetry feature
GET	/api/twin/health-score	ditto_sync	the health feature
GET	/api/twin/events?n=20	ditto_sync (with a doc store)	recent events from MongoDB
POST	/api/twin/external	data	accepts a push from another twin

POST	/api/chat	intelligent	streams the diagnostic agent's reply
------	-----------	-------------	--------------------------------------

The `DittoSyncService` registers itself under `ditto_sync`, so the first five routes work as soon as you register it as shown above. The last two need the data router registered under `data` and the multi-agent system under `intelligent`; you add those registrations when you wire up those layers.

`/health` doubles as the twin's self-report. Alongside `asset_id` and `status` it returns a `counters` object — running totals of the things a healthy twin should have none of, such as failed storage writes, dropped telemetry items, and failed event-bus publishes. A monitoring system that scrapes this endpoint sees a data-path problem as a climbing counter long before it would surface as a gap in a chart.

Locking the API with a key

Everything under `/api/` — including your own routes and the streaming endpoints below — sits behind one shared secret. Set it and every request must present it:

```
DT_SECURITY__API_KEY=<a long random string>
```

Ordinary clients send the key as a header:

```
curl -H "x-api-key: $KEY" http://localhost:8500/api/twin/state
```

Two kinds of client cannot set headers: the browser's `EventSource` (used for SSE) and WebSocket connections opened from a page. For those, the same key goes in the URL as a query parameter — `/api/viz/stream?api_key=...` — and the framework accepts either form. The dashboard does this for you automatically (chapter 13); you only meet it when writing your own streaming client.

A few paths stay open by design: `/health` (so load balancers and `dyon infra check` can probe liveness), the OpenAPI docs, and the dashboard shell itself (the data behind the dashboard still needs the key — an unauthenticated visitor gets an empty app). With no key configured, the middleware is not installed at all and the API behaves as one open surface, which is the right shape for local development and the wrong one for anything exposed — production mode (chapter 03) therefore refuses to start without a key. The key comparison is constant-time, so it cannot be guessed by timing responses.

Adding your own routes

Your asset will have endpoints the framework cannot anticipate. Build them on a standard FastAPI `APIRouter` and attach it to the app:

```
from fastapi import APIRouter

my_router = APIRouter()

@my_router.get("/api/pump/curve")
async def pump_curve():
    return {"head_m": 32.0, "flow_lpm": 120.0}

app = create_app(config, registry)
app.include_router(my_router)
```

Streaming with Server-Sent Events

Most of the API follows the ordinary request–response pattern: a client asks, the server answers, the connection closes. That suits anything that changes slowly, where polling every few seconds is fine.

The `/api/chat` endpoint is different. It talks to the reasoning tier’s diagnostic agent (chapter 08), and an LLM produces its answer gradually, so the endpoint streams the reply using **Server-Sent Events** (SSE) — one HTTP connection the server pushes text into as it is generated, so the answer appears piece by piece instead of arriving all at once. Each piece is sent as a `data: {"chunk": "..."}` line, and a final `data: [DONE]` marks the end.

```
const response = await fetch("/api/chat", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({message: "Why is the temperature rising?", stream:
true}),
});

const reader = response.body.getReader();
while (true) {
  const {value, done} = await reader.read();
  if (done) break;
  const chunk = JSON.parse(new TextDecoder().decode(value).replace("data: ",
""));
  if (chunk.chunk) process.stdout.write(chunk.chunk);
}
```

The same SSE mechanism is well suited to live telemetry too — one persistent connection delivering updates beats the browser opening a fresh request every second. For the chat endpoint to do anything, the multi-agent system must be registered under `intelligent`, which brings us to the tier that gives the twin something worth saying. The next chapter, though, stays with the simpler kind of response: the control tier, which acts on readings without any reasoning at all.

07 — The Agent Layer: Control

The Agent layer is where the twin acts, and this chapter covers the floor it stands on: the tier that acts without thinking. It checks readings against rules, moves the twin through a small set of states, and can run a control loop — all deterministically, in milliseconds, giving the same answer to the same input every time. That predictability is the whole point. A safety alarm must fire the instant a reading crosses a danger line, so it cannot wait on an LLM that takes seconds to respond and may reason slightly differently each call. The reasoning and governance tiers above handle novelty and nuance; this tier handles speed and certainty, and they are complementary rather than competing.

The control tier offers three tools: a threshold engine with a built-in state machine, a configurable multi-state engine for richer assets, and a PID controller for continuous regulation.

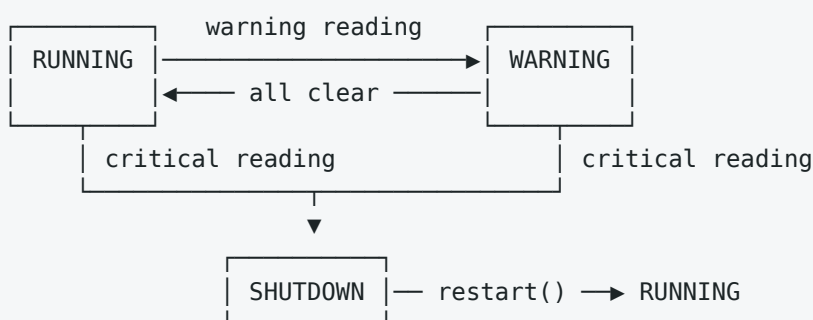
The ThresholdRuleEngine and its state machine

You met this engine in chapter 02. It evaluates your sensor thresholds (and any custom rules) on a timer, and uses the result to drive a finite state machine.

```
from dyon.reactive import ThresholdRuleEngine

ThresholdRuleEngine(self.config, self.bus,
                    ts_store=ts, cache=cache, doc_store=doc,
                    custom_rules=[],          # optional, see below
                    eval_interval=5)         # evaluate every N seconds
```

A finite state machine is simply a system that is always in exactly one of a fixed set of states and moves between them only along defined transitions — like a traffic light that is red, amber, or green but never two at once, and never skips the sequence. The engine's machine has three states and moves between them like this:



On each evaluation, the engine counts how many fields sit in their critical zone and how many sit in their warning zone, then moves the machine: any critical field sends it to `shutdown`; otherwise a warning field promotes `running` to `warning`; otherwise, if nothing is in warning, `warning` falls back to `running`.

The asymmetry at the bottom is deliberate and worth restating, because it is easy to expect symmetry that is not there. The drop into `warning` clears itself once readings recover, but the drop into `shutdown` does not. A twin that tripped on a critical fault stays shut down until something explicitly calls `restart()`.

— because a tripped asset should not silently come back to life the moment a reading flickers into range. You decide when to restart, whether from an operator action, the API, or the governance tier.

Every state change does two things on its own: it writes an event to MongoDB and publishes a `state.changed` event on the bus. The current state is always in Redis, so any layer can read it instantly:

```
state = cache.get_state()          # "running" | "warning" | "shutdown"
state = reactive_layer.get_state() # same, from the engine object
```

And when a maintenance workflow or test needs to move the machine by hand, the transitions are plain synchronous methods:

```
reactive_layer.restart()          # shutdown → running
reactive_layer.recover()          # warning → running
reactive_layer.shutdown_asset()   # force a shutdown
```

Custom rules

Thresholds catch single fields crossing single limits. For anything more — a rate of change, a combination of fields — you write a custom rule. A rule is any object with a `rule_name` and an `evaluate(readings)` method that returns `"warning"`, `"critical"`, or `None`:

```
class RateOfChangeRule:
    """Flag temperature rising too fast, even while it is still in range."""
    rule_name = "temp_roc_alarm"

    def evaluate(self, readings: dict) -> str | None:
        roc = readings.get("temperature_c_roc") # produced by the data-
management loop
        if roc is None:
            return None
        if roc > 5.0:
            return "critical"
        if roc > 2.0:
            return "warning"
        return None

class CorrelationRule:
    """Flag low pressure and low flow occurring together."""
    rule_name = "low_pressure_low_flow"

    def evaluate(self, readings: dict) -> str | None:
        pressure, flow = readings.get("pressure_bar"),
readings.get("flow_rate_lpm")
        if pressure is None or flow is None:
            return None
        return "critical" if (pressure < 3.0 and flow < 80.0) else None
```

The `readings` dictionary the engine passes in holds the latest value of every sensor field, including the smoothed and rate-of-change signals the data layer computed. Pass your rules in alongside the thresholds, and a rule returning `"critical"` counts exactly like a critical threshold breach:

```
ThresholdRuleEngine(self.config, self.bus, ts_store=ts, cache=cache,
doc_store=doc,
                    custom_rules=[RateOfChangeRule(), CorrelationRule()])
```

For actions beyond moving the state machine, the package also offers two small, reusable action objects you can call from your own rule or event handler: `LogEventAction`, which records a document event, and `PublishMQTTAction`, which sends a message to an MQTT topic.

Richer assets: the MultiStateFSMRuleEngine

Three states are enough for a simple alarm, but many assets have more operating modes than running, warning, and shutdown. `MultiStateFSMRuleEngine` lets you define any set of states and the logic that chooses between them. You subclass it, declare the states and their severities, and implement one method — `compute_desired_state()` — which looks at the readings and returns the state the twin should be in:

```
from dyon.reactive import MultiStateFSMRuleEngine

class PumpFSM(MultiStateFSMRuleEngine):
    _states = ["NOMINAL", "DEGRADING", "CRITICAL", "SENSOR_FAULT"]
    _initial_state = "NOMINAL"
    _severity_map = {
        "DEGRADING": "warning",
        "CRITICAL": "critical",
        "SENSOR_FAULT": "warning",
    }

    def compute_desired_state(self, readings: dict) -> str | None:
        vib = readings.get("vibration_mm_s")
        if vib is None:
            return "SENSOR_FAULT"
        if vib > 5.0:
            return "CRITICAL"
        if vib > 2.5:
            return "DEGRADING"
        return "NOMINAL"
```

That single method is all you write. The base class generates a transition into each state, so `compute_desired_state()` can move the twin to any state directly, and it handles the rest — performing the transition, logging the change, updating Redis, and publishing the event. Return a state name to request a move, or `None` to stay put. The `_severity_map` decides the severity attached to each state when it is entered; anything you leave out defaults to `warning`. The engine evaluates every fifteen seconds by default.

Escalating to the reasoning tier

Most state changes are routine and the control tier handles them alone — a field crosses a line, the state moves, an alarm is recorded. Some, though, deserve investigation rather than just an alarm. When the multi-state engine enters a state whose severity is `warning` or `critical`, it publishes a second event alongside `state.changed`: a `reactive.escalation_requested` event carrying a ready-made question about what caused the change.

```
# published automatically – you never call this yourself
DomainEvent(
    event_type="reactive.escalation_requested",
    source_layer="reactive",
    source_asset=self.config.asset_id,
    payload={
        "from_state": "NOMINAL",
        "to_state": "DEGRADING",
        "severity": "warning",
        "question": "The reactive FSM transitioned from NOMINAL to DEGRADING ..."
    },
    "What is driving this state change and what action is recommended?",
)

```

The reasoning tier (chapter 08) subscribes to this event and begins a targeted investigation immediately, instead of waiting for its own next polling cycle. This is the seam between the two tiers: the control tier decides that something changed, in milliseconds, and hands the question of why to the tier built to reason about it. You can subscribe to either event yourself:

```
self.bus.subscribe("state.changed", on_state_change)
self.bus.subscribe("reactive.escalation_requested", on_escalation)
```

Continuous control: the PIDController

Some assets need more than discrete states — they need a value held steadily at a target. A PID controller does that. PID stands for proportional–integral– derivative, a feedback method that answers “the value is at X, I want it at Y, how hard should I push right now?” The proportional term reacts to the current gap, the integral term corrects a persistent offset that lingers, and the derivative term eases off as the gap closes quickly so the system does not overshoot.

`PIDController` reads a process variable from InfluxDB and publishes a control command to the twin’s MQTT control topic, which the asset (or its simulator) subscribes to:

```
from dyon.reactive import PIDController

PIDController(self.config, self.bus,
               ts_store=ts, mqtt_transport=mqtt,
               process_variable="pressure_bar", # the field to hold steady
               setpoint=4.2, # the target value
               output_min=1000.0, output_max=2000.0,
               control_key="speed_rpm", # key in the published command
               kp=2.0, ki=0.5, kd=0.1,
               sample_time=1.0) # compute every N seconds

```

Each cycle it publishes a message like `{"speed_rpm": 1523.4}` to `dt/{asset_id}/control`. You can move the target at runtime:

```
pid_layer.setpoint = 4.5
```


A PID loop and a state machine often run together: the PID handles the fast, continuous fine-tuning (every second), while a state machine or the governance tier above it makes the slower, strategic calls about whether to be regulating at all.

Versioned rules: the RuleRepository

By default, rules live in your Python code and change only when you redeploy. When you need to add, change, or roll back rules on a running twin — and keep an audit trail of which version was active when — `RuleRepository` stores them as versioned rows in PostgreSQL. (It needs the `PostgresAdapter` from chapter 04, and a Postgres instance you supply, since `infra up` does not provision one.)

```
from dyon.reactive.rule_repository import RuleRepository, PersistedRule
from dyon.data.storage.postgres import PostgresAdapter
import os

pg = PostgresAdapter(os.environ["POSTGRES_DSN"])
await pg.connect()

repo = RuleRepository(pg, asset_id=config.asset_id)
await repo.setup() # creates the dt_rules table if needed

await repo.upsert(PersistedRule(
    rule_name="high_temp_rule",
    condition_type="threshold",
    params={"threshold": 70.0, "direction": "high"},
    severity="critical",
))

rules = await repo.load_active_rules() # the current active set
history = await repo.history("high_temp_rule") # every version, oldest first
await repo.deactivate("high_temp_rule") # stop loading it, keep it for audit
```

Calling `upsert()` again with the same `rule_name` increments its version rather than overwriting, so the history is preserved.

There is one wrinkle to connect this to the engine. A rule's `evaluate()` is synchronous and cannot `await` a database read, so you do not query Postgres from inside it. Instead, a small background task keeps an in-memory snapshot fresh, and the rule reads the snapshot:

```
class HotReloadRule:
    rule_name = "dynamic_high_temp"

    def __init__(self, repo: RuleRepository, target="high_temp_rule"):
        self._repo, self._target, self._snapshot = repo, target, None

    async def refresh_forever(self, interval=30):
        while True:
            active = await self._repo.load_active_rules()
            self._snapshot = next((r for r in active if r.rule_name ==
self._target), None)
            await asyncio.sleep(interval)

    def evaluate(self, readings: dict) -> str | None:
```

```
    rule = self._snapshot
    if rule is None:
        return None
    temp = readings.get("temperature_c")
    if temp is not None and temp > rule.params.get("threshold", 70.0):
        return rule.severity
    return None

rule = HotReloadRule(repo)
asyncio.create_task(rule.refresh_forever())
engine = ThresholdRuleEngine(config, bus, ts_store=ts, cache=cache,
                             doc_store=doc, custom_rules=[rule])
```

The engine calls `evaluate()` synchronously every cycle; the background task keeps the parameters current. No blocking, no nested event loops.

The control tier gives the twin reflexes. But a reflex only knows that something is wrong, never why. When the engine escalates a state change, it is asking a question it cannot answer itself — and the next chapter builds the layer that can.

08 — The Agent Layer: Reasoning

The control tier can tell you that a reading crossed a line. This tier tells you why. It is slower — seconds to minutes rather than milliseconds — because it reasons about the meaning of the data rather than its bare value, and that is exactly what some faults require. A combination of three readings that are each individually normal, a trend only visible against last month's, a situation whose significance depends on context no rule author foresaw: rules cannot catch these, but a reasoning agent armed with domain knowledge can.

The tier has two halves that work together. A **knowledge graph** holds your asset's domain expertise — its components, the ways it fails, and what to do about each. A **diagnostic agent** wraps a large language model with tools that read the twin's live data and query that graph. The graph supplies facts specific to your asset that a general-purpose LLM was never trained on; the LLM supplies language, reasoning, and the judgement to decide which facts it needs.

The knowledge graph

A knowledge graph stores relationships between concepts as first-class objects, not rows in tables. Instead of recording a temperature reading and writing code to join it to a fault, you store the relationship itself

```
— (Symptom: high_temperature) <-[:CAUSES]- (FailureMode: overheating) -[:REQUIRES]->
  (MaintenanceAction: check_cooling)
```

— and then traverse it: from the symptom a reading triggers, follow the edges to every failure mode it implies, the components each affects, and the actions each calls for. That multi-hop traversal is awkward to express as SQL joins or as rules, and natural as a graph.

The framework's graph models four kinds of thing and the links between them:

- **Components** — the physical parts of the asset (bearing, seal, impeller);
- **Failure modes** — what can go wrong (overheating, cavitation, wear);
- **Symptoms** — the sensor signals that point to a failure mode;
- **Maintenance actions** — what a technician should do in response.

You declare all of this as plain Python dataclasses; the framework translates them into Neo4j for you, so you never write Cypher for the standard pattern:

```
from dyon.intelligent import KnowledgeGraph, KnowledgeGraphSpec, FailureMode,
SymptomMapping
from neo4j import GraphDatabase

driver = GraphDatabase.driver(config.neo4j.uri,
                             auth=(config.neo4j.user, config.neo4j.password))

spec = KnowledgeGraphSpec(
    components=["bearing", "seal", "impeller", "motor", "shaft"],
    failure_modes=[
        FailureMode(name="overheating", severity="high",
                    maintenance_actions=["check_cooling_system",
                                         "inspect_bearing"],
                    affected_components=["bearing", "motor"]),
```

```

        FailureMode(name="cavitation", severity="high",
                    maintenance_actions=["check_inlet_pressure", "reduce_speed"],
                    affected_components=["impeller"]),
        FailureMode(name="bearing_wear", severity="medium",
                    maintenance_actions=["lubricate_bearing", "replace_bearing"],
                    affected_components=["bearing"]),
    ],
    symptom_mappings=[
        # A symptom triggers when its field crosses the threshold in the given
        # direction.
        SymptomMapping(symptom_name="high_temperature",
                        sensor_field="temperature_c",
                        threshold=60.0, direction="high",
                        failure_modes=["overheating", "bearing_wear"]),
        SymptomMapping(symptom_name="low_pressure", sensor_field="pressure_bar",
                        threshold=3.5, direction="low",
                        failure_modes=["cavitation"]),
        SymptomMapping(symptom_name="high_vibration",
                        sensor_field="vibration_mm_s",
                        threshold=2.5, direction="high",
                        failure_modes=["bearing_wear", "cavitation"]),
    ],
)

kg = KnowledgeGraph(config, driver)
kg.setup_from_spec(spec)  # builds the graph; idempotent, safe to re-run

```

You can query the graph directly. Given current readings, it returns the active symptoms; given symptoms, it returns the implicated failure modes with their actions, ranked so the most severe come first:

```

readings = {"temperature_c": 72.0, "pressure_bar": 3.2, "vibration_mm_s": 1.2}

kg.diagnose_from_readings(readings)
# ['high_temperature', 'low_pressure']

kg.diagnose(['high_temperature', 'low_pressure'])
# [{'failure': 'overheating', 'severity': 'high',
#   'actions': ['check_cooling_system', 'inspect_bearing']},
#  {'failure': 'cavitation', 'severity': 'high',
#   'actions': ['check_inlet_pressure', 'reduce_speed']}]

kg.get_components()
# ['bearing', 'seal', 'impeller', 'motor', 'shaft']

```

If your domain needs node types beyond this equipment-health pattern — routes, depots, suppliers — add raw Cypher through the spec's `custom_cypher` field, which runs after the standard schema is built. A logistics twin, for instance, can use this to add `Depot` and `Route` nodes:

```

spec = KnowledgeGraphSpec(
    components=[...], failure_modes=[...], symptom_mappings=[...],
    custom_cypher=[
        "MERGE (:Depot {code: 'DEP-04'})",
        "MERGE (:Route {name: 'north_loop'})",
    ],
)

```

```
1,
)
```

Every statement uses `MERGE`, which is idempotent, so the graph stays consistent across restarts.

The diagnostic agent

On its own, a language model is just text in, text out — it knows nothing about your sensors right now. An agent fixes that by giving the model **tools**: Python functions it can choose to call mid-reasoning. You ask a question; the model decides it needs data and calls a tool; the framework runs the function and hands back the result; the model reads it, decides whether it needs more, and eventually writes its answer. The model directs the information-gathering instead of being handed everything up front, which is both cheaper and easy to extend — add a tool and the model can use it without any retraining.

`DiagnosticAgent` comes with five tools wired to the twin’s live stores:

Tool	What it returns
<code>get_twin_state</code>	the full Ditto Thing
<code>get_sensor_readings</code>	the latest value of every sensor field
<code>diagnose_asset</code>	active symptoms and failure modes from the graph
<code>get_recent_events</code>	the last N events from MongoDB
<code>get_asset_components</code>	the asset’s components from the graph

You build it from the config — `build_llm()` reads your `DT_LLM__*` settings and returns the right LangChain model — and hand it the stores and graph it needs:

```
from dyon.intelligent import DiagnosticAgent
from dyon.intelligent.agent import build_llm

agent = DiagnosticAgent(config, llm=build_llm(config),
                        ditto_client=ditto, ts_store=ts, doc_store=doc,
                        knowledge_graph=kg)

answer = await agent.ask("Why is the pump running hot?")
# "Temperature is 72 °C, above the 60 °C warning threshold. The knowledge graph
# links this to two failure modes – overheating (high) and bearing wear (medium).
# Recommended actions: check the cooling system and inspect the bearing."
```

The agent chooses which tools to call based on the question, so the same agent handles “Is anything wrong?”, “What maintenance should I schedule?”, and “How does current vibration compare to the last day?”

Every call the agent makes is bounded, because an LLM provider is a remote service that can hang, ramble, or flake, and a twin must keep running when it does. Three `DT_LLM__*` settings enforce this: `timeout_s` caps each request (default 60 s), `max_tokens` caps each reply (default 2048 — also your cost ceiling per call), and `max_retries` absorbs transient provider errors (default 2). On top of the per-request timeout, `ask()` itself has a hard deadline covering the whole tool-calling loop, so even a pathological conversation ends with an error string rather than a stuck coroutine. A slow provider therefore costs you one answer, never the layer.

One boundary is deliberate and worth stating plainly: the agent is a **diagnostician, not an actuator**. It can read data and advise, but it cannot open a valve, publish a control command, or change the state machine. Acting on its findings is the job of the governance tier in chapter 09, which keeps every consequential action behind one place that can enforce safety limits — rather than letting several agents reach for the controls independently.

How an agent thinks: observe, reason, act

A `TwinAgent` works in three steps that mirror a careful analyst. `observe()` gathers raw facts — sensor values, recent history, the FSM state — without drawing conclusions. `reason()` interprets them, typically by calling the LLM, and produces findings. `act()` carries out consequences, such as logging or alerting.

The split matters because the steps run on different conditions. `observe()` and `reason()` run every cycle, so an agent always has a current read on the situation to show a dashboard, even when all is well. `act()` runs only when `observe()` reports an anomaly, so the side effects fire only when they should. The stock `DiagnosticAgent` observes passively (it never flags an anomaly by itself); you add detection by subclassing it:

```
class ActiveDiagnosticAgent(DiagnosticAgent):
    async def observe(self) -> dict:
        readings = {f: self._ts_store.get_latest(f) for f in
self.config.field_names}
        symptoms = self.kg.diagnose_from_readings(readings)
        return {"anomaly_detected": len(symptoms) > 0,
                "symptoms": symptoms, "readings": readings}
```

For the MAS (below) to record an agent's status well, have `reason()` return `action`, `severity`, and `summary` keys alongside anything domain-specific.

Adding your own tools

To give an agent a tool the framework cannot provide — a call into a domain simulator, a query to an external service — override `_build_extra_tools()`. It is invoked while the agent is being built, so any state the tool needs must be set on `self` before you call `super().__init__()`:

```
from langchain_core.tools import tool as lc_tool

class PumpDiagnosticAgent(DiagnosticAgent):
    def __init__(self, *args, curve_model=None, **kwargs):
        self._curve = curve_model          # set before the parent compiles
tools
        super().__init__(*args, **kwargs)

    def _build_extra_tools(self) -> list:
        curve = self._curve
        @lc_tool
        def expected_head(flow_lpm: float) -> dict:
            """Return the pump's expected head for a given flow rate."""
            return {"head_m": curve.head_at(flow_lpm)}
        return [expected_head]
```

Running agents continuously: the MultiAgentSystem

A single `ask()` answers one question. To have the reasoning tier watch the asset on its own, wrap your agents in a `MultiAgentSystem` and return it from `build_layers()`:

```
from dyon.intelligent import MultiAgentSystem

"intelligent": MultiAgentSystem(self.config, self.bus,
                                agents=[agent],
                                monitor_interval=30, # run the loop every N
seconds
                                max_concurrent=1,    # parallel LLM calls
allowed
                                cache=cache,         # live status in Redis
                                doc_store=doc,       # durable findings in
MongoDB
```

On each cycle the MAS runs every agent's `observe()` and `reason()`, and calls `act()` for any agent whose observation flagged an anomaly — also publishing an `agent.action` event when it does. Agents are tried in priority order, highest first.

`max_concurrent` caps how many agents may call the LLM at once. The default of `1` serialises them, which keeps a local model (Ollama) or a rate-limited API from being overwhelmed; raise it only when your provider can take the load.

The two stores serve different needs and can be used together. With `cache`, the MAS writes a compact status record to Redis under `mas_agent_<agent_name>` on every cycle, so a dashboard always sees the latest:

```
{"agent_name": "diagnostic", "domain": "general", "anomaly": true,
 "action": "recommend_inspection", "severity": "warning",
 "detail": "temperature 72 °C, overheating likely", "ts_s": 1748400000}
```

With `doc_store`, it writes to MongoDB only when a cycle is noteworthy — an anomaly, or an action other than routine monitoring — recording both that compact status (`mas_agent_<name>`) and a fuller record with observations, findings, and tool calls (`mas_agent_detail_<name>`). Routine cycles are not persisted, so the history stays clean and queryable:

```
doc.get_events_by_type("mas_agent_diagnostic", n=100)
doc.get_events_by_type("mas_agent_detail_diagnostic", n=100)
```

Responding to escalations immediately

The MAS does not only run on its timer. When it starts, it subscribes to the `reactive.escalation_requested` event from chapter 07. The moment the control tier's state machine enters a warning or critical state, that event arrives carrying a ready-made investigation question, and the MAS routes it straight to its highest-priority agent instead of waiting for the next polling cycle. The agent's answer is logged to MongoDB as `mas_escalation_response`. This is the chain end to end:

```
FSM enters a warning/critical state
  → reactive.escalation_requested (with a question)
  → MAS routes it to the top agent
  → the agent investigates with its tools
```

- the answer is logged as `mas_escalation_response`
- the governance tier reads it on its next cycle

Feeding the governance tier

After every cycle the MAS keeps each agent's full observation and findings in memory, reachable by name. The governance tier reads this during its own observe phase to fold the agents' diagnoses into its decisions:

```
detail = mas.get_agent_detail("diagnostic") # latest observations + findings
```

It can also pose a fresh question to a specific agent on demand, without disturbing the monitoring loop:

```
answer = await mas.ask_agent("diagnostic",  
                             "Is the rising vibration consistent with bearing  
wear?")
```

Choosing and skipping pieces

The LLM is chosen entirely through configuration — no code change moves you between providers:

```
DT_LLM_PROVIDER=anthropic # openai | anthropic | ollama  
DT_LLM_MODEL=claude-sonnet-4-6  
DT_LLM_API_KEY=sk-ant-...  
# Ollama needs no key: DT_LLM_PROVIDER=ollama, DT_LLM_BASE_URL=http://  
localhost:11434
```

And if you want an agent without a knowledge graph, construct a `KnowledgeGraph` and simply never call `setup_from_spec`. Its `diagnose_asset` tool reports no symptoms and `get_asset_components` returns an empty list, while everything else — the sensor and event tools, the reasoning — works as normal.

The twin can now explain itself. The final active layer takes the next step: it reads these explanations and decides what the asset should actually do.

09 — The Agent Layer: Governance

The control tier responds to readings; the reasoning tier explains them. Neither decides, at a strategic level, what the asset should do — keep running, call for a human, ask another twin for data, or take control action itself. That is this tier’s job, and it is the top of a single twin.

It works as an **OODA loop** — observe, orient, decide, act — running every few seconds. Each pass gathers the situation, makes sense of it, chooses an action, and carries it out. The four steps map directly onto four methods you can see and override, and the whole loop is built so that the riskier a decision is, the more control sits with deterministic safety rules rather than with the AI.

The loop, step by step

Observe takes a snapshot from the layers below — the FSM state, the health score, the latest reading of every field, the most recent events, and the current findings of each MAS agent. It only gathers; it draws no conclusions.

Orient makes sense of that snapshot. At minimum a `GoalPlanner` reads it and returns a structured risk assessment. If you wire in an `AutonomousOverseer` (below), an LLM does this step instead, reading the agent findings and producing a reasoned decision. Either way, orient is where “temperature is 72 °C” becomes “this asset is at high risk and needs attention.”

Decide turns the assessment into a concrete plan, through a strict three-level hierarchy that we will return to, because it is the heart of the design.

Act executes the plan — escalating to a human, shutting the asset down, querying a peer twin, or running one step of an RL policy — and caches a one-line summary of the cycle to Redis under `ooda_last_cycle` so a dashboard can show what the twin just did.

Orient with rules: the GoalPlanner

The `GoalPlanner` is the simplest way to do the orient step. It is not an LLM and not a learner — it is a plain rule-based assessor. You give it goals, which are declarations of intent, and it reads the observation and reports whether the situation threatens them:

```
from dyon.autonomous import GoalPlanner, Goal

planner = GoalPlanner(goals=[
    Goal(name="prevent_shutdown", description="Avoid the shutdown state",
    priority=20),
    Goal(name="maintain_health", description="Keep health above 80", priority=10),
])
```

Its `assess()` reads the health score, the state, and the count of recent critical events, and returns a risk level plus a few flags the decide step acts on:

Situation	risk_level	flag set
running, health $\geq$ 75	low	—

running, health 50–75	medium	—
warning, or health < 50	high	requires intervention
shutdown	critical	requires human intervention
three or more recent critical events	critical	requires human intervention

The planner notices threats; it does not optimise toward the goals. Working out the best action to achieve them is what an RL policy does, later in this chapter. To tailor the assessment to your asset, subclass it and extend `assess()`:

```
class PumpGoalPlanner(GoalPlanner):
    async def assess(self, observation: dict) -> dict:
        result = await super().assess(observation)
        t = observation.get("telemetry", {})
        temp, pressure = t.get("temperature_c"), t.get("pressure_bar")
        if temp and pressure and temp > 65.0 and pressure < 3.8:
            result["risk_level"] = "critical"
            result["requires_human_intervention"] = True
            result["reason"] = "combined thermal and pressure degradation"
        return result
```

Decide: the safety hierarchy

`decide()` is deliberately layered so that the AI never overrules safety. It works through three levels in order and stops at the first that produces an action.

The first level is **hard safety constraints** from the assessment. If it flags that a human is required, the plan is `request_human` and nothing else is consulted. The same is true for a required shutdown or a need for external data. These come from deterministic rules and cannot be talked out of.

The second level is the **overseer's strategic decision**, if you wired one in. As long as no safety constraint fired, an overseer action other than `no_action` becomes the plan.

The third level is **RL tactical control**. With no constraint and no overseer action, if a policy is loaded and the risk is `low`, the plan is `rl_control` — the strategic question is already settled, so the policy simply optimises the output. Otherwise the cycle ends in `maintain_current`, and nothing happens.

The shape is intentional: safety rules first, strategic reasoning second, learned control last. The RL policy can never take an action the safety level would forbid, and it is never asked to reason about strategy it was not trained for.

Each plan maps to a concrete effect in `act()`:

Plan	Produced when	Effect
<code>request_human</code>	the assessment requires human intervention	logs a critical event, publishes <code>autonomous.human_requested</code> , and calls the notifier if one is wired in
<code>shutdown</code>	the assessment requires a shutdown	calls <code>reactive.shutdown_asset()</code> and logs it
<code>query_peer</code>	the assessment needs external data	uses a connector to query the named peer twin

<overseer action>	the overseer chose a non- no_action action	publishes autonomous.<action> and logs it, for domain code to act on
rl_control	a policy is loaded and risk is low	runs one inference step of the policy
maintain_current	none of the above	nothing

Wiring the loop together

The OODA loop needs references to the lower layers it observes and acts on, so you build those first inside `build_layers()`, keep the references, and pass them in:

```
from dyon.autonomous import OODALoop, GoalPlanner, Goal

def build_layers(self):
    ts, doc, cache = InfluxAdapter(self.config), MongoAdapter(self.config),
    RedisAdapter(self.config)
    ditto = DittoClient(self.config)

    router = TelemetryRouter(self.config, self.bus, ts_store=ts, doc_store=doc,
    cache=cache)
    reactive = ThresholdRuleEngine(self.config, self.bus, ts_store=ts,
    cache=cache, doc_store=doc)
    mas = MultiAgentSystem(self.config, self.bus, agents=[...], cache=cache,
    doc_store=doc)
    planner = GoalPlanner([Goal(name="maintain_health", description="...",
    priority=10)])

    ooda = OODALoop(self.config, self.bus,
                    ts_store=ts, cache=cache, doc_store=doc, ditto_client=ditto,
                    models={}, reactive=reactive, mas=mas, connectors=[],
                    planner=planner,
                    policy=None,          # optional PolicyDeployer
                    overseer=None,       # optional AutonomousOverseer
                    loop_interval=5)

    return {"data": router, "network": MQTTIngestor(self.config, self.bus,
    router=router),
            "reactive": reactive, "intelligent": mas, "autonomous": ooda}
```

With just a planner, this gives you a twin that escalates to a human when health drops or it shuts down, and otherwise holds steady. The next two sections add the two optional brains: an LLM overseer for strategy, and an RL policy for control.

Orient with an LLM: the AutonomousOverseer

When a rule-based assessment is too blunt, an `AutonomousOverseer` does the orient step with a language model. It reads every MAS agent's findings, can interrogate specific agents for more detail, and returns a structured decision with its full reasoning recorded for audit. Because it has the whole picture — and, in a multi-twin system, can reach across twins — it can act on patterns no single agent sees.

You give it the LLM, the twin's MAS, the goals, and the vocabulary of actions it may choose:

```

from dyon.autonomous.overseer import AutonomousOverseer
from dyon.intelligent.agent import build_llm

overseer = AutonomousOverseer(
    config=config, llm=build_llm(config), mas=mas,
    goals=["maintain_health", "prevent_shutdown"],
    available_actions=["reduce_load", "schedule_maintenance", "no_action"],
)

ooda = OODALoop(..., overseer=overseer)

```

Each cycle, the overseer formats the observation and the agent findings into a prompt, optionally calls its `query_mas_agent` tool for deeper analysis, and must call its `submit_decision` tool to return an `OverseerDecision` — an `action`, the `reasoning` behind it, a `risk_level`, the goals it addresses, and the agent queries it made along the way. If it errors or never submits, it returns a safe `no_action` default, so the loop always has a valid decision and the safety level in `decide()` still governs.

`available_actions` is a hard vocabulary, not a suggestion. If the model submits an action outside the list — a hallucinated verb, a rephrasing, a typo — the decision is rejected and the rejection is fed back to the model, which gets another attempt within its iteration budget; a model that never produces a valid action falls through to the same `no_action` default. So the only strings that can ever leave the overseer as `autonomous.<action>` events are ones you wrote yourself, which is what makes it safe for `act()` to dispatch on them. The prompt is defended in the other direction too: sensor values, event text, and anything else that originates outside the twin is fenced off as data in the prompt, so a malicious string arriving through telemetry cannot restate the overseer's instructions.

Every overseer decision is logged to MongoDB as `ooda_overseer_decision` during the orient step — before any safety gating — so the audit trail records what the overseer actually recommended even when a constraint overrides it. For a multi-twin system, pass an `extra_query_fn` so the overseer can question agents in other twins:

```

async def cross_twin_query(agent_name: str, question: str) -> str:
    for other_mas in (pump_b.mas, manifold.mas):
        for agent in other_mas.agents:
            if agent.agent_name == agent_name:
                return await other_mas.ask_agent(agent_name, question)
    return f"Agent '{agent_name}' not found."

overseer = AutonomousOverseer(config, build_llm(config), mas,
                               goals=[...], available_actions=[...],
                               extra_query_fn=cross_twin_query)

```

When the overseer's chosen action reaches the act step, the loop publishes it as an `autonomous.<action>` event and logs it; to actually drive an actuator, override `act()` in your OODA subclass to handle your action vocabulary.

Control with a learned policy: RL

The third level of `decide()` runs a reinforcement-learning policy — a small neural network that maps the current readings to a control output. Unlike the PID controller of chapter 07, which follows a fixed formula, an RL policy learns its strategy by trial and error against a simulated model, which lets it handle

non-linear dynamics a hand-tuned formula struggles with. The trade-off is that it must be trained first and is not interpretable.

Training happens against a Gymnasium environment that wraps one of your simulation models. You declare what the agent sees (`obs_fields`), the single output it adjusts (`control_field`), and the value it is trying to hold at a target:

```
import numpy as np
from dyon.autonomous import GenericTwinEnv, PolicyTrainer

env = GenericTwinEnv(
    model=physics_model,          # a TwinModel from chapter 05
    control_field="speed_rpm",    # the output the agent sets
    process_variable="pressure_bar", # the value it regulates
    target=4.2,                  # the value it aims for
    ctrl_min=800.0, ctrl_max=2200.0, # the physical range of the output
    obs_fields=["temperature_c", "pressure_bar", "flow_rate_lpm"],
    obs_low=np.array([0.0, 0.0, 0.0]),
    obs_high=np.array([100.0, 10.0, 300.0]),
    max_steps=500,
)
```

Each step, the agent sees the observation vector, outputs an action that is scaled into the control range, the model advances, and the agent receives a reward. By default the reward is `-abs(process_variable - target)` — zero when it hits the target, increasingly negative the further off it is — so over many episodes the learner shapes the network toward actions that keep the value on target.

You train with `PolicyTrainer`, choosing one of four algorithms (SAC is the default and a good choice for continuous control; TD3, PPO, and A2C are also available):

```
trainer = PolicyTrainer(env, algorithm="SAC", save_dir="./policies")
trainer.train(total_timesteps=200_000)
trainer.save("pump_pressure_policy") # → ./policies/pump_pressure_policy.zip
```

The CLI wraps the same flow: `dyon train --timesteps 200000 --algorithm SAC --save pump_pressure_policy`, loading the environment from a module that exposes an `env` object.

To run the trained policy live, hand a `PolicyDeployer` to the OODA loop. When the decide step returns `rl_control`, the deployer reads the observation fields from InfluxDB, runs one deterministic forward pass, scales the result, and publishes the control command to MQTT:

```
from dyon.autonomous import PolicyDeployer

policy = PolicyDeployer(
    policy_path="./policies/pump_pressure_policy.zip",
    config=self.config, ts_store=ts, mqtt_transport=mqtt,
    obs_fields=["temperature_c", "pressure_bar", "flow_rate_lpm"],
    control_field="speed_rpm", ctrl_min=800.0, ctrl_max=2200.0,
    algorithm="SAC",
)
ooda = OODALoop(..., policy=policy)
```

If you need more than distance-to-target, pass your own `reward_fn(obs, target)` when building the environment — for example, tracking the target while penalising high temperature:

```
def shaped_reward(obs: dict, target: float) -> float:
    pressure = obs.get("sim_pressure_bar", 0.0)
    temp      = obs.get("sim_temperature_c", 100.0)
    return -abs(pressure - target) - max(0.0, temp - 60.0) * 0.1

env = GenericTwinEnv(..., reward_fn=shaped_reward)
```

`PolicyDeployer` can also run a policy you already hold in memory rather than load from a file — pass it as `policy=...` instead of a path. That is the hook the next section depends on.

When you cannot write the reward

Everything above assumes you can express what “good” means as a reward function. Often you cannot — when the behaviour you want is a human skill that is easier to demonstrate than to score. For those cases the framework includes a whole toolkit, `dyon.learning`, that learns from demonstrations instead: it can clone an expert’s actions directly, or recover the reward function implicit in their behaviour and feed it back into the same `GenericTwinEnv` and `PolicyDeployer` you have just seen. That is the subject of chapter 12.

Escalating to humans

When the loop decides `request_human`, it publishes an `autonomous.human_requested` event and, if you gave it a `HumanNotifier`, sends a message through every backend the notifier holds:

```
from dyon.notifications.notifier import HumanNotifier, SlackBackend, EmailBackend

notifier = HumanNotifier(backends=[
    SlackBackend(webhook_url=os.getenv("SLACK_WEBHOOK")),
    EmailBackend(smtp_host="smtp.example.com", smtp_port=587,
                 sender="dyon@example.com", recipients=["oncall@example.com"],
                 username=os.getenv("EMAIL_USER"),
                 password=os.getenv("EMAIL_PASS")),
])

ooda = OODALoop(..., notifier=notifier)
```

The framework ships `SlackBackend`, `EmailBackend`, and `WebhookBackend`, and a notifier sends to all of them at once; pass an empty list to stay silent (useful in tests). For anything more bespoke, subscribe to `autonomous.human_requested` on the bus and handle it yourself.

That completes a single self-managing twin: it senses, models, exposes, reacts, diagnoses, and decides. The next chapter steps outside one twin — to how twins talk to each other, and how many of them combine into one.

10 — Connectors and Collection Twins

Every chapter so far has built one twin in isolation. Real systems are rarely one asset, though — a fleet of pumps, a plant where one stage feeds the next, a water network where a failure ripples downstream. This chapter covers the two tools for working with many twins at once: **connectors**, which let one twin reach another, and **collection twins**, which treat a whole group as a single twin built on top of those connectors.

Connectors: how twins reach each other

A connector is a transport between twins. The framework ships three, each operating at a different layer and suited to a different need:

Connector	Layer	Mechanism	Use it to
DittoConnector	services	HTTP to another twin's Eclipse Ditto record	read another twin's state
MQTTConnector	data	publish/subscribe on a shared MQTT broker	stream data between twins
APIConnector	intelligent	HTTP to another twin's <code>/api/chat</code>	ask another twin's agent

You collect a twin's connectors in a `ConnectorRegistry`, registering each under the layer it exposes:

```
from dyon.connector import ConnectorRegistry, DittoConnector, MQTTConnector,
APIConnector

registry = ConnectorRegistry(config)
registry.register(DittoConnector(config)) # read
peers' state
registry.register(MQTTConnector(config, known_twins=["pump_002", "pump_003"])) #
stream to peers
registry.register(APIConnector({"pump_002": "http://localhost:8502", # ask
peers' agents
                                "pump_003": "http://localhost:8503"}))
```

To use one, ask the registry for a route to the target twin at the layer you need, then call `query` (request a response) or `push` (send one-way):

```
# read pump_002's telemetry through Ditto
conn = registry.find_route("pump_002", layer="services")
telemetry = await conn.query("pump_002", {"feature": "telemetry"})

# push a value into pump_002's data layer over MQTT
conn = registry.find_route("pump_002", layer="data")
await conn.push("pump_002", {"external_pressure": 3.9})

# ask pump_002's diagnostic agent a question
conn = registry.find_route("pump_002", layer="intelligent")
```



```
answer = await conn.query("pump_002", {"question": "Are you experiencing any faults?"})
```

The choice of layers is deliberate. A twin exposes its data stream (Data, through MQTT), its state (Services, through Ditto), and its reasoning (the Agent layer, through the chat API — the registry keys it by the module name, `intelligent`). It exposes nothing else, and the two omissions are the point.

The Simulation and Model layer is never exposed, because a peer that reached into it would be coupled to the internals of your model; a peer that needs a prediction asks the Services layer for one instead. And the Agent layer exposes its reasoning interface but never its control surface, since letting one twin trip another's deterministic control would be unsafe, and anything worth knowing about that control is already published through Ditto's `health` and `telemetry` features. Sharing state and reasoning, but never raw models or control, is what keeps one twin from reaching in and destabilising another.

Collection twins: many twins as one

A collection twin does not model a physical asset. It is a coordinator that sits above a group of component twins, reaches them through connectors, and makes group-level decisions. There are four patterns, each matching a different way twins relate to one another:

Pattern	Use when	Example
AggregateDT	the twins are identical and you want one fused state	a fleet of identical pumps
CollectionDT	the twins are alike but you want each tracked individually	a bank of sensors
CompositeDT	one twin's output feeds the next	boiler → turbine → generator
NetworkDT	the twins are linked physically and failures propagate	a water distribution network

All four share the same lifecycle: you construct them with the component IDs and a connector registry, and call `run(interval=...)` to drive their orchestration loop.

AggregateDT — one fused view

`AggregateDT` fuses identical twins into a single state. Each cycle it reads every member's health and telemetry and reports the averages, the worst case, and how many members it could reach:

```
from dyon.collection import AggregateDT

fleet = AggregateDT(collection_id="pump_fleet_001", config=fleet_config,
                    component_twin_ids=["pump_001", "pump_002", "pump_003"],
                    connector_registry=registry, ditto_client=ditto)

state = await fleet.aggregate_state()
# {"avg_health": 87.5, "min_health": 62.0, "worst_state": "warning",
#  "member_count": 3, "active_count": 3,
#  "telemetry_summary": {"temperature_c": {"mean": 48.2, "min": 44.1, "max":
#  72.0}, ...}}

results = await fleet.broadcast_command({"setpoint_change": {"pressure_bar":
4.5}})
# {"pump_001": True, "pump_002": True, "pump_003": False}
```


It publishes its fused state to its own Ditto feature, and raises an `aggregate.critical` event when the worst member has shut down.

CollectionDT — a group with individual identity

`CollectionDT` keeps each member distinct, which is what you want when the point is to find the odd one out. It offers outlier detection and health ranking over the group:

```
from dyon.collection import CollectionDT

bank = CollectionDT(collection_id="sensor_bank_a", config=group_config,
                    component_twin_ids=[f"sensor_{i:03d}" for i in range(1, 51)],
                    connector_registry=registry)

outliers = await bank.find_outliers("temperature_c", z_threshold=2.0)
# ['sensor_017', 'sensor_031'] – members more than 2 std-devs from the group mean

ranking = await bank.rank_by_health()
# [('sensor_017', 42.0), ('sensor_031', 58.0), ...] – worst first
```

CompositeDT — a chain of stages

`CompositeDT` models a system where one stage's output is the next stage's input. A list of `BoundaryCondition`s declares those data flows, and the composite pushes each source field into the matching target field every cycle:

```
from dyon.collection import CompositeDT, BoundaryCondition

plant = CompositeDT(collection_id="power_plant_001", config=plant_config,
                    component_twin_ids=["boiler_001", "turbine_001",
                    "generator_001"],
                    connector_registry=registry,
                    hierarchy={"power_plant_001": ["boiler_001", "turbine_001",
                    "generator_001"]},
                    boundary_conditions=[
                        BoundaryCondition(source_twin="boiler_001",
                        source_field="steam_pressure",
                        target_twin="turbine_001",
                        target_field="inlet_pressure"),
                        BoundaryCondition(source_twin="turbine_001",
                        source_field="shaft_speed_rpm",
                        target_twin="generator_001",
                        target_field="input_speed_rpm"),
                        # transform the value in transit – by a callable, or
                        "scale"/"offset"
                        BoundaryCondition(source_twin="boiler_001",
                        source_field="steam_temp_c",
                        target_twin="turbine_001",
                        target_field="inlet_temp_c",
                        transform=lambda x: x * 0.95), # 5%
                        pipe loss
                    ],
                    ditto_client=ditto)
```

A `BoundaryCondition` passes its value through unchanged by default; set `transform` to a callable, or to `"scale"` (with `transform_factor`) or `"offset"` (with `transform_offset`), to adjust it in transit.

Because the composite delivers boundary values into the target's data layer, the target twins must be reachable through a registered data-layer connector.

The composite also supports hot-swapping a stage — after maintenance, say:

```
await plant.swap_component("turbine_001", "turbine_002_refurbished")
```

By default the swap is validated: the replacement must already expose its Ditto features and produce every field the boundary conditions read from it, or the swap raises and leaves the composite untouched.

NetworkDT — a graph with cascade effects

NetworkDT connects twins by typed relationships and reasons about the graph. You declare the edges, and it can find which failing twin would harm the most others downstream, and which twin is a single point of failure:

```
from dyon.collection import NetworkDT, TwinRelationship

network = NetworkDT(collection_id="water_district_north", config=net_config,
                    component_twin_ids=["pump_001", "tank_001", "valve_002",
                    "pump_003"],
                    connector_registry=registry,
                    relationships=[
                        TwinRelationship("pump_001", "tank_001", "feeds",
                        {"capacity_lpm": 200}),
                        TwinRelationship("tank_001", "valve_002", "supplies",
                        {"pipe_mm": 150}),
                        TwinRelationship("valve_002", "pump_003", "feeds"),
                    ],
                    neo4j_driver=driver) # optional: persists the topology in
Neo4j

risks = await network.detect_cascade_risk()
# [{"twin_id": "pump_001", "health": 38.0,
#   "downstream_affected": ["tank_001", "valve_002", "pump_003"],
#   "cascade_severity": 3}]

bottlenecks = await network.find_bottlenecks()
# ['tank_001'] — removing it would disconnect the network
```

Each cycle it logs the network's overall health and publishes a `network.cascade_risk` event for the most severe risk it finds.

Running several collections together

When more than one collection watches the same components — an `AggregatedDT` for the fused view and a `NetworkDT` for cascade analysis — a `CollectionOrchestrator` runs them concurrently and shuts them down together:

```
from dyon.collection.orchestrator import CollectionOrchestrator

orchestrator = CollectionOrchestrator(interval=15)
orchestrator.add(fleet)
```

```
orchestrator.add(network)
asyncio.run(orchestrator.run_forever())
```

You now have every building block the framework offers, from a single sensor reading up to a coordinated network of twins. The next chapter puts a full selection of them together in one worked example, end to end.

11 — A Full Worked Example

This chapter assembles everything the guide has covered into one complete, runnable twin: a centrifugal pump with all four layers, a physics model, a knowledge graph, and a diagnostic agent. Nothing here is new — it is the pieces from chapters 02 through 09 wired together in a single `build_layers()`. Read it as a template. To turn it into a twin of your asset you change only three things: the sensor fields, the physics equations, and the knowledge-graph contents. The framework code never changes.

The project is three small files plus configuration:

```
my_pump_twin/
├── .env           - identity and backend addresses
├── twin.py        - the twin: sensors, model, graph, and build_layers()
├── run.py         - the entry point
└── simulate.py    - synthetic readings for testing
```

twin.py

The file has four parts. First the sensors, which define everything the layers above will store and watch (chapter 03). Note `flow_rate_lpm` and `speed_rpm` carry no thresholds — they feed the model but are not alarmed on.

```
import asyncio
import numpy as np
from dotenv import load_dotenv
from neo4j import GraphDatabase

from dyon.core.config import TwinConfig, SensorFieldSpec
from dyon.core.base import AbstractDigitalTwin
from dyon.data import InfluxAdapter, MongoAdapter, RedisAdapter
from dyon.data.writer import TelemetryRouter
from dyon.data.management import DataManagementPipeline
from dyon.network import MQTTIngestor
from dyon.simulation import ODEModel, ModelRunner
from dyon.services.ditto.client import DittoClient
from dyon.services.ditto.sync import DittoSyncService
from dyon.reactive import ThresholdRuleEngine
from dyon.intelligent import (
    KnowledgeGraph, KnowledgeGraphSpec, FailureMode, SymptomMapping,
    DiagnosticAgent, MultiAgentSystem,
)
from dyon.intelligent.agent import build_llm
from dyon.autonomous import OODALoop, GoalPlanner, Goal

load_dotenv()

# 1 - Sensors
config = TwinConfig(sensor_fields=[
    SensorFieldSpec(name="temperature_c", nominal=45.0, noise_std=0.3,
```

```

        warn_threshold=60.0, crit_threshold=75.0,
        threshold_direction="high"),
        SensorFieldSpec(name="pressure_bar", nominal=4.2, noise_std=0.05,
                        warn_threshold=3.5, crit_threshold=2.5,
                        threshold_direction="low"),
        SensorFieldSpec(name="flow_rate_lpm", nominal=120.0, noise_std=2.0),
        SensorFieldSpec(name="vibration_mm_s", nominal=1.5, noise_std=0.1,
                        warn_threshold=2.5, crit_threshold=5.0,
                        threshold_direction="high"),
        SensorFieldSpec(name="speed_rpm", nominal=1450.0, noise_std=5.0),
    ])

```

Second the physics model, whose predictions the simulation and model layer compares against reality to produce residuals (chapter 05):

```

# 2 – Physics model
class PumpODE(ODEModel):
    model_name = "pump_physics"

    def derivatives(self, t, y, u):
        T, P, Q = y
        n = u / 1450.0
        Q_heat = (1.0 - 0.75) * abs(P * Q) / 600.0
        dT = (Q_heat - (T - 25.0)) / 120.0
        dP = (4.2 * n**2 - P) / 5.0
        dQ = (120.0 * n - Q) / 3.0
        return [dT, dP, dQ]

physics_model = PumpODE(
    initial_state=np.array([25.0, 4.2, 120.0]),
    state_names=["temperature_c", "pressure_bar", "flow_rate_lpm"],
    control_field="speed_rpm", nominal_input=1450.0,
)

```

Third the knowledge graph, which gives the diagnostic agent the failure knowledge to reason with (chapter 08):

```

# 3 – Knowledge graph
kg_spec = KnowledgeGraphSpec(
    components=["bearing", "seal", "impeller", "motor", "shaft"],
    failure_modes=[
        FailureMode(name="overheating", severity="high",
                    maintenance_actions=["check_cooling_system",
                    "inspect_bearing"],
                    affected_components=["bearing", "motor"]),
        FailureMode(name="cavitation", severity="high",
                    maintenance_actions=["check_inlet_pressure", "reduce_speed"],
                    affected_components=["impeller"]),
        FailureMode(name="bearing_wear", severity="medium",
                    maintenance_actions=["lubricate_bearing", "replace_bearing"],
                    affected_components=["bearing"]),
    ],
    symptom_mappings=[
        SymptomMapping(symptom_name="high_temperature",

```

```

sensor_field="temperature_c",
            threshold=60.0, direction="high",
            failure_modes=["overheating", "bearing_wear"]),
    SymptomMapping(symptom_name="low_pressure", sensor_field="pressure_bar",
                    threshold=3.5, direction="low",
                    failure_modes=["cavitation"]),
    SymptomMapping(symptom_name="high_vibration",
                    sensor_field="vibration_mm_s",
                    threshold=2.5, direction="high",
                    failure_modes=["bearing_wear", "cavitation"]),
],
)

```

Fourth the twin itself. This is where everything comes together. The pattern is the one chapter 09 introduced: build the storage adapters once, build the control and reasoning tiers and keep references, then hand those references to the OODA loop so it can observe and act on them.

```

# 4 – The twin
class PumpTwin(AbstractDigitalTwin):
    def build_layers(self):
        ts = InfluxAdapter(self.config)
        doc = MongoAdapter(self.config)
        cache = RedisAdapter(self.config)
        ditto = DittoClient(self.config)

        driver = GraphDatabase.driver(self.config.neo4j.uri,
                                      auth=(self.config.neo4j.user,
                                             self.config.neo4j.password))
        kg = KnowledgeGraph(self.config, driver)
        kg.setup_from_spec(kg_spec)

        router = TelemetryRouter(self.config, self.bus, ts_store=ts,
                                doc_store=doc, cache=cache)
        reactive = ThresholdRuleEngine(self.config, self.bus, ts_store=ts,
                                       cache=cache, doc_store=doc)

        agent = DiagnosticAgent(self.config, llm=build_llm(self.config),
                                ditto_client=ditto, ts_store=ts, doc_store=doc,
                                knowledge_graph=kg)
        mas = MultiAgentSystem(self.config, self.bus, agents=[agent],
                               monitor_interval=60, cache=cache, doc_store=doc)

        planner = GoalPlanner(goals=[
            Goal("uptime", "Avoid entering shutdown state", priority=20),
            Goal("health", "Keep health score above 80", priority=10),
        ])
        ooda = OODALoop(self.config, self.bus,
                        ts_store=ts, cache=cache, doc_store=doc,
                        ditto_client=ditto,
                        models={"pump_physics": physics_model},
                        reactive=reactive, mas=mas, connectors=[]),
                        planner=planner)

        return {

```

```

        "network":      MQTTIngestor(self.config, self.bus, router=router),
        "data":         router,
        "data_mgmt":    DataManagementPipeline(self.config, self.bus,
ts_store=ts, cache=cache),
        "simulation":   ModelRunner(self.config, self.bus, ts_store=ts,
models=[physics_model]),
        "services":     DittoSyncService(self.config, self.bus, ts_store=ts,
cache=cache,
                                     doc_store=doc, ditto_client=ditto),

        "reactive":     reactive,
        "intelligent":  mas,
        "autonomous":   ooda,
    }

```

The dictionary keys set the order in which layers initialise — `network` and `services` first so the MQTT subscription and the Ditto Thing exist before the rest relies on them, and `autonomous` last. Once running, every layer loops concurrently.

run.py

```

import asyncio
from dotenv import load_dotenv
from dyon.core.lifecycle import TwinLifecycle
from twin import PumpTwin, config

load_dotenv()
lifecycle = TwinLifecycle()
lifecycle.add(PumpTwin(config))
asyncio.run(lifecycle.run_forever())

```

simulate.py

With no hardware attached, the simulator from chapter 02 feeds the twin. Here it runs normally for thirty seconds, then injects a critical bearing fault — high temperature and high vibration together:

```

import asyncio
from dotenv import load_dotenv
from dyon.physical.simulator import GenericSimulator
from twin import config

load_dotenv()

async def main():
    sim = GenericSimulator(config, publish_interval=1.0)

    async def inject_later():
        await asyncio.sleep(30)
        print("Injecting bearing fault...")
        sim.inject_fault({"temperature_c": 78.0, "vibration_mm_s": 6.0}) # both
above critical

```

```
await asyncio.gather(sim.run(), inject_later())

asyncio.run(main())
```

.env

```
DT_ASSET_ID=pump_001
DT_ASSET_TYPE=centrifugal_pump
DT_ASSET_NAME=Plant A Pump

DT_MQTT__BROKER=localhost
DT_INFLUX__URL=http://localhost:8086
DT_INFLUX__TOKEN=my-super-secret-token
DT_INFLUX__ORG=digital_twin
DT_INFLUX__BUCKET=asset_telemetry
DT_MONGO__URI=mongodb://admin:password@localhost:27017
DT_REDIS__URL=redis://localhost:6379
DT_DITTO__URL=http://localhost:8080
DT_NE04J__URI=bolt://localhost:7687
DT_NE04J__PASSWORD=password

DT_LLM__PROVIDER=anthropic
DT_LLM__MODEL=claude-sonnet-4-6
DT_LLM__API_KEY=sk-ant-...
```

The credentials here are the dev-mode defaults that match what `dyon infra up` provisions — right for this local walkthrough, and exactly what production mode (chapter 03) would refuse to start with. A deployed version of this twin swaps each of them for a real secret, sets `DT_SECURITY__MODE=production`, and gains an API key on every route (chapter 06) as part of the same switch.

Running it, and what you will see

Start the infrastructure, run the twin in one terminal and the simulator in another:

```
dyon infra up --layers network,data,services,intelligent
dyon infra check
python run.py          # terminal 1
python simulate.py     # terminal 2
```

On startup the layers announce themselves, then the fault propagates up the stack:

```
INFO dyon.twin.pump_001          - Initialising layer: network
INFO dyon.service_ditto.pump_001 - Ditto gateway ready (HTTP 200)
INFO dyon.service_ditto.pump_001 - Ditto Thing 'org.example:pump_001' ready
INFO dyon.intelligent.pump_001   - Knowledge graph built: 5 components, 3 failure
modes
INFO dyon.network.pump_001       - MQTT ingestor listening on 'dt/pump_001/
telemetry'
INFO dyon.simulation.pump_001    - ModelRunner started with models:
['pump_physics']
```



```

INFO dyon.reactive.pump_001      - ThresholdRuleEngine started (interval=5s)
INFO dyon.intelligent.pump_001   - MAS started with 1 agents (max_concurrent=1)
INFO dyon.autonomous.pump_001    - OODA autonomous loop started (interval=5s)

# ... 30 seconds in, the fault arrives ...
WARNING dyon.reactive.pump_001   - State: running → shutdown
WARNING dyon.autonomous.pump_001 - Human intervention requested: Asset is in
shutdown state

```

Trace that last sequence against the chapters. The router stored the faulty reading (chapter 04); the threshold engine saw two fields go critical and drove the state machine to `shutdown` (chapter 07); the OODA loop’s planner read that shutdown state as critical risk requiring a human, and the act step logged the request and would notify anyone you wired in (chapter 09). The twin reasoned its way from a raw reading to an escalation, across four layers, with no domain code beyond the three declarations at the top of `twin.py`.

One thing to expect: `shutdown` latches. Even if you clear the fault, the twin stays shut down until something calls `reactive.restart()`, exactly as chapter 07 described — a tripped pump should not restart itself.

Adapting it to your asset

To change...	Edit...
sensor names, nominals, thresholds	<code>sensor_fields</code> in <code>TwinConfig</code>
the physics	<code>derivatives()</code> in your <code>ODEModel</code> subclass
the state vector	<code>initial_state</code> and <code>state_names</code>
components, failure modes, symptoms	the <code>KnowledgeGraphSpec</code>
what counts as a threat	the <code>GoalPlanner</code> goals (or a subclass)
identity and backends	<code>.env</code>

That is the whole framework in one asset. The next chapter steps beyond it, to a twin whose right behaviour cannot be written as a rule or a reward — and must be learned from example.

12 — Learning from Demonstration

Chapter 09 trained an RL policy against a reward you wrote by hand. That works when “good behaviour” is something you can score with a formula. Often it is not. The behaviour you want may be a human skill — an operator nursing a temperamental machine, a pilot judging a crosswind landing — that an expert can demonstrate far more easily than anyone can describe. This chapter is about learning from those demonstrations, using the `dyon.learning` package.

A policy, recall, is just a function from state to action. There are three ways to obtain one, and they differ in what they need:

- **Reinforcement learning** needs a reward function and discovers a policy by trial and error. Use it when you can write the reward.
- **Imitation learning** needs demonstrations and produces a policy that copies the expert. Use it when you have expert data and simply want to reproduce it.
- **Inverse RL** needs demonstrations and an environment, and recovers the reward function the expert seems to be following — and, with it, a policy. Use it when the reward is the hard part.

The insight behind the third option is that for nuanced skills, the reward is the thing you cannot write down. Inverse RL recovers it from behaviour instead of asking you to guess. Everything in this chapter is domain-agnostic: the same tools that teach a robot arm a trajectory teach a dispatcher when to reroute a fleet.

The inverse-RL and adversarial trainers wrap the `imitation` library, which pins `gymnasium 0.29.x` and `stable-baselines3 2.2.x`. Those bounds are set in `pyproject.toml`; upgrading past them breaks `imitation`.

Describing the observation: FeatureSpec

Whatever you learn is only as good as the features it sees, and a subtle bug class in this kind of work is the observation vector and its declared space drifting out of step. `FeatureSpec` removes that risk by defining the observation once and deriving both the numeric vector and the matching `Gymnasium Box` from the same column list:

```
from dyon.learning import FeatureSpec, Scalar, Categorical

spec = FeatureSpec(version=1, columns=[
    Scalar("temperature", lambda c: c["temperature"], 0.0, 100.0),
    Scalar("pressure", lambda c: c["pressure"], 0.0, 10.0),
    Categorical("mode", lambda c: c["mode"], ["idle", "run", "fault"],
one_hot=True),
])

box = spec.box() # Box of shape (len(spec),)
vec = spec.encode({"temperature": 55.0, "pressure": 4.2, "mode": "run"})
```

Each column’s extractor reads from whatever context you pass to `encode` — a dict, a session object, anything. `Scalar` clips to its bounds so a stray reading cannot violate the space, and `Categorical`

encodes one-hot or as a normalised index. The spec carries a `version` tag every recorded demonstration with it, so a trainer can refuse data captured under a different layout. A long-lived twin leans on this heavily: its observations are versioned precisely so that recordings made before a sensor was added are never mixed with those made after.

The common data format: Demonstrations

Every algorithm here consumes the same bundle — arrays of transitions plus the spaces they were recorded in:

```
from dyon.learning import Demonstrations

demos = Demonstrations(
    obs=obs_array,          # (N, obs_dim)
    acts=act_array,         # (N,) discrete, or (N, act_dim) continuous
    next_obs=next_obs_array, # (N, obs_dim)
    dones=done_array,       # (N,) bool — episode-final flags
    observation_space=env.observation_space,
    action_space=env.action_space,
    episode_ids=ep_array,    # optional — lets it regroup into trajectories
)
```

You feed demonstrations in through a `DemonstrationSource`. The framework provides `ArrayDemonstrationSource` for arrays you already hold and `CorpusDemonstrationSource` for a versioned `.npz` in a `TrainingCorpus`. Where your demonstrations live somewhere else — a twin that has been logging its own transitions to MongoDB, say — subclass `DemonstrationSource` and load them there, filtering by feature version so a stale layout can never slip in. When you need a held-out set for validation, `split_demonstrations(demos, holdout_frac=0.2)` provides one.

Copying the expert: behavioural cloning

The simplest method is behavioural cloning — plain supervised learning that maps each demonstrated state to the action the expert took. It needs no environment, only the demonstrations and their spaces. The trained policy is hosted inside an SB3 PP0 container so it saves and loads exactly like the RL and IRL policies, and deploys through the same `PolicyDeployer`:

```
from dyon.learning import BCTrainer

bc = BCTrainer(demonstrations=demos, save_dir="./policies")
bc.train(n_epochs=20)
bc.save("cloned_policy")          # → ./policies/cloned_policy.zip
model = bc.get_sb3_model()        # use this to warm-start an IRL generator
```

Cloning is fast and makes a strong baseline, but it has a known weakness: compounding error. In a state no expert ever visited, the policy has no signal and can drift further and further off course. When you have an interactive expert — an oracle or simulator that can label new states on demand — `DAGgerTrainer` fixes this by interleaving the learner's own rollouts with fresh expert labels for the states it actually reaches.

Recovering the reward: AIRL

The headline method is adversarial inverse RL. `AIRLTrainer` trains a generator policy against a discriminator that learns to tell expert transitions from generated ones, using the environment only for its dynamics — the reward itself is learned. When training finishes you get two artifacts: a policy and a reusable reward network.

```
from dyon.learning import AIRLTrainer

airl = AIRLTrainer(env=env, demonstrations=demos,
                  init_policy=bc.policy,          # warm-start from BC: faster,
                  steadier
                  allow_variable_horizon=True)    # for variable-length episodes
airl.train(total_timesteps=100_000)

reward = airl.reward_fn()      # the recovered reward, as a LearnedRewardFn
paths  = airl.save("airl_policy") # writes the policy .zip and the reward .pt
```

`GAILTrainer` shares the same interface but its discriminator is not a reusable reward — reach for it when you want the strongest imitation and do not need the reward back. `MaxEntIRLTrainer` is a classic, dependency-free alternative that recovers a simpler linear reward.

The recovered reward is where inverse RL earns its keep, because it transfers even when the policy cannot. `LearnedRewardFn` makes it behave like any other reward — you can optimise a fresh policy against it, even under new dynamics, with no new demonstrations:

```
from dyon.learning import LearnedRewardFn
from stable_baselines3 import PPO

reward = LearnedRewardFn.load("./policies/airl_policy_reward.pt")

wrapped = reward.wrap_env(env)          # overlay it on a vectorised env
model = PPO("MlpPolicy", wrapped).learn(50_000) # train a new policy on it

# or drop it straight into the framework's generic RL env from chapter 09:
env = GenericTwinEnv(..., reward_fn=reward.make_generic_reward_fn(obs_fields))
```

The whole flow at once: the skill-transfer pipeline

These stages compose into a single repeatable operation, and `SkillTransferPipeline` chains them: behavioural cloning to warm-start, AIRL to recover the reward and refine, then ordinary RL to optimise that learned reward.

```
from dyon.learning import SkillTransferPipeline

pipe = SkillTransferPipeline(env=env, demonstration_source=my_source,
                           save_dir="./policies",
                           corpus=training_corpus,    # optional: versions
                           results
                           dataset_name="my_skill")

result = pipe.run(bc_epochs=20, airl_timesteps=100_000,
```

```

        final_rl_timesteps=50_000,          # 0 to stop after AIRL
        allow_variable_horizon=True)
# result.model, result.reward, result.stages_run

```

Every stage is optional, which means the same object also handles reward reuse: point it at a saved reward and run only the final RL stage to retrain under new conditions.

```

pipe.run(bc_epochs=0, airl_timesteps=0,
        reward_path="./policies/my_skill_reward.pt",
        final_rl_timesteps=50_000, final_rl_from_scratch=True)

```

Crucially, the pipeline does not deploy blindly. Score a candidate on held-out demonstrations first, and promote it only if it beats the incumbent:

```

train, holdout = split_demonstrations(my_source.load(), 0.2)
result = pipe.run(demos=train, bc_epochs=20, airl_timesteps=100_000)
result.metrics = pipe.evaluate(result.model, holdout, result.reward)
# {"action_match": 0.81, "mean_return": 23.4}

if pipe.should_promote(result.metrics, current_metrics):
    pipe.version(result)          # push the policy and reward to the corpus
    deploy(result.model)

```

Versioning through the corpus buys integrity as well as bookkeeping. Policy and reward files are pickle-format under the hood (SB3 zips, torch checkpoints), and unpickling a file executes code inside it — so a twin must never load one it cannot trust. The corpus closes that loop: every `push_version` records a SHA-256 checksum of the artifact in the dataset's manifest, every download verifies the bytes against it, and a mismatch deletes the file and raises `IntegrityError` before anything deserializes it. What a twin loads is therefore byte-identical to what your trusted trainer uploaded, even if the object store sits on shared infrastructure.

`action_match` is the fraction of held-out expert actions the policy reproduces (for discrete actions); `mean_return` is the average return under the learned reward. Finally, `SyncTrigger` decides when to re-run all of this — when enough new demonstrations have accrued, or an interval has passed — so a twin can keep improving from fresh expert data on its own:

```

from dyon.learning import SyncTrigger

trigger = SyncTrigger(demo_threshold=100, interval_seconds=86_400)
fire, reason = trigger.should_fire(current_demo_count)
if fire:
    run_pipeline_and_maybe_promote()
    trigger.mark_fired(current_demo_count)

```

Run that check from a background task on your twin — every few hours, or whenever a batch of demonstrations lands — and the circle closes on its own, from expert demonstration to retrained policy to deployed skill.

Deploying a learned policy

Because every learner here saves an SB3 `.zip`, a learned policy deploys through the same `PolicyDeployer` from chapter 09. Pass `algorithm="BC"` so it loads through the PPO container, or hand it a policy object you already hold:

```
from dyon.autonomous import PolicyDeployer

policy = PolicyDeployer(policy_path="./policies/cloned_policy.zip",
                        config=self.config, ts_store=ts, mqtt_transport=mqtt,
                        obs_fields=[...], control_field="...",
                        ctrl_min=..., ctrl_max=..., algorithm="BC")
ooda = OODALoop(..., policy=policy)
```

`PolicyDeployer` suits twins that control a continuous variable. For twins whose “action” is a discrete, context-driven choice — a dispatcher choosing a route, for instance — you wrap the trained model in a small adapter that builds the observation from your own state and maps the action index back to a domain action. The adapter is the only domain-specific part; the trained policy underneath is the same object either way.

Which method to reach for

- You can write a good reward → plain RL (chapter 09).
- You have expert data and want to copy it quickly → `BCTrainer`.
- Cloning drifts and you have an oracle → `DaggerTrainer`.
- You want the strongest imitation and don’t need the reward → `GAILTrainer`.
- The reward is the hard part, or you want to reuse it → `AIRLTrainer`.
- You want the full, validated, self-improving loop → `SkillTransferPipeline` with `SyncTrigger`.

That covers everything the twin can do. The final chapter turns to how a person sees it: a live dashboard, built from the same config, that puts a face on all of it.

13 — Visualization

Everything the twin knows by now lives behind an API. Readings are in InfluxDB, state is in Redis, the diagnostic agent answers on `/api/chat`, and the services layer (chapter 06) already noted that its Server-Sent Events mechanism “is well suited to live telemetry too.” This chapter follows that thread to its conclusion: a live, in-browser dashboard that puts a human face on the twin — a 3D (or schematic) view of the asset that recolours itself as the asset’s state changes, trend charts and KPI tiles that update in real time, a panel that shows the multi-agent system working, and a conversational interface you can type or speak to that answers from live data and draws charts on request.

The dashboard is a first-class part of the framework, but it is **entirely opt-in**. A twin that never asks for it behaves exactly as it did in every previous chapter. You turn it on with a single flag or a single function call, and from that point on the framework reads your existing `TwinConfig` and builds a working dashboard with no further authoring.

The one-line dashboard

You already build the twin’s web API with `create_app()` (chapter 06). To get a dashboard, pass `include_viz=True`:

```
from dyon.services.api import create_app

app = create_app(config, registry, include_viz=True)
```

That single flag mounts two things onto the same app: a set of read-only routes under `/api/viz/*` that serve the dashboard’s data, and the static dashboard client itself at `/dashboard`. Open `http://localhost:8000/dashboard/` and you are looking at your twin. Leave the flag off — its default — and not one byte of the app changes.

If you would rather keep the dashboard in its own process, or point a dashboard at a twin that is already running, the CLI serves the bundled client for you:

```
dyon dashboard --api http://localhost:8000
```

This serves the dashboard’s static files locally and points them at the twin’s API, then opens your browser. The twin needs no extra wiring: in dev mode, mounting the dashboard opens CORS on the twin’s API so a client served from anywhere can read it; in production mode (chapter 03) the twin answers only the origins named in `DT_SECURITY__CORS_ORIGINS`, so list the dashboard’s origin there. Use `--port` to change the local port and `--no-open` to skip launching the browser.

When the twin has an API key set (chapter 06), the dashboard authenticates itself without help: open it as `/dashboard/?api_key=<key>` once and the client remembers the key in the browser’s local storage, sending it as a header on ordinary requests and as a query parameter on its live streams — the two forms the twin accepts. Without the key, the dashboard shell loads but every panel stays empty.

For full control you can also mount the dashboard onto any FastAPI app you build yourself:


```
from dyon.visualization import mount_visualization

mount_visualization(app, config, registry)
```

`mount_visualization` discovers the twin’s time-series store, document store, multi-agent system, and event bus from the service registry, so a fully wired twin needs no further arguments. Everything `create_app(..., include_viz=True)` does, it does by calling this function.

What you get for free

The dashboard is built from a **dashboard specification** — a description of which panels to show and what each one is bound to. You do not have to write that specification. The framework derives a complete one from your `TwinConfig`, reading the same `sensor_fields` you declared back in chapter 03. From a config with a `temperature_c` field (warn at 60, critical at 75) and a `pressure_bar` field (warn at 3, critical at 2, low direction), the derived dashboard gives you:

- **A KPI tile per sensor field**, showing the latest value, its unit, and a small sparkline of recent history. A tile turns amber or red when its reading crosses the warning or critical threshold you set on the field.
- **Trend charts**, one per unit, plotting every field that shares that unit on a common axis so the scales line up.
- **An alarm banner** driven by your fields’ thresholds — the same warn/critical levels the control tier already uses, surfaced visually.
- **A scrolling event log** of the twin’s recent events.
- **A chat panel** wired to the diagnostic agent, so an operator can ask the twin questions without leaving the page.
- **An agents panel** that shows the multi-agent system in action — one card per agent, updating as each agent observes, reasons, and acts.

Crucially, the thresholds and directions on the dashboard are your thresholds. The framework reads `warn_threshold`, `crit_threshold`, and `threshold_direction` straight off each `SensorFieldSpec`; a field you marked as alarming when it falls too low (like pressure) colours and alarms in the right direction without you saying so twice.

How the dashboard is laid out

The client is not a flat wall of widgets. It is an **app shell**: a sidebar on the left for navigating between views, a stage in the middle that holds the active view, and a chat dock that slides in from the right (and a floating chat button to summon it). The shell carries a light/dark theme toggle, persisted in the browser; a spec can also ship its own `theme` tokens to brand the dashboard.

A single twin’s view inside that stage is **tabbed**, so the most important thing is always the first thing you see:

- **Home** foregrounds the asset itself — the 3D (or schematic) view — alongside a live **agent-chart canvas** and an embedded chat box. This is the at-a-glance view: what the asset looks like right now, what the agent has drawn, and a place to ask a question.
- **Telemetry** holds the detail: the KPI tiles, the trend charts (the first promoted to a larger “hero” chart), the alarm banner, the event log, and any finite-state-machine panel.
- **Agents** shows the multi-agent system working — covered below.

You do not arrange any of this. The client routes each panel from the derived spec into the region and tab where it belongs, by `kind`, so any twin's spec lays out sensibly regardless of how its panels are ordered.

The specification is the contract

The derived dashboard is a normal Python object you can inspect and change before mounting it. `derive_default_spec(config)` returns a `DashboardSpec`; adjust it, then hand it to `mount_visualization` as the `spec` argument:

```
from dyon.visualization import derive_default_spec, mount_visualization

spec = derive_default_spec(config)
spec.voice_enabled = True          # turn on the microphone button
spec.asset_name = "Plant A - Pump 001"

mount_visualization(app, config, registry, spec=spec)
```

Because the dashboard is just data, you can reorder panels, drop ones you do not want, or add your own. Every panel is a `PanelSpec` with a `kind` (`kpi`, `chart`, `alarms`, `events`, `fsm`, `chat`, `scene`, or `html`) and a small config dictionary. The client renders each kind from a registry of components, and you can override or extend that registry from the browser side too (`DyonDash.register("kpi", fn)`) — but for most twins, deriving the spec and nudging a few fields is all you will ever do.

Live updates come from the event bus

The dashboard's tiles, charts, and 3D view move because the framework bridges the twin's **event bus** straight to the browser. Every layer in this guide publishes domain events — the telemetry router emits `telemetry.routed` on every reading, the control tier emits state changes — and the visualization layer subscribes to the bus and forwards those events to the browser over Server-Sent Events. The client opens one persistent connection to `/api/viz/stream` and updates in place as events arrive, reconnecting on its own if the connection drops.

This is the same SSE mechanism chapter 06 described for chat, now carrying live telemetry, and it is completely generic: it works for any twin's events because it keys off the event bus every twin already has. You wrote no streaming code to get a live dashboard — publishing events, which your layers already do, is the whole of the contract. When the dashboard first loads it also asks `/api/viz/history` for recent readings so the charts open already populated, then keeps them current from the live stream.

The conversational interface

The chat — embedded on the Home tab and available everywhere through the dock — posts to a single `/api/chat` endpoint backed by one conversational agent, so an operator can ask “why is the temperature climbing?” and read the agent's reasoning streamed back token by token. Two things make it more than a text box.

It can draw. The framework ships a ready-made chat agent for exactly this role. `make_dashboard_chat_agent` builds a diagnostic agent (chapter 08) already equipped with chart and forecast tools, so it answers in prose and draws charts when asked. Hand it to the dashboard and you are done:

```

from dyon.services.api import create_app
from dyon.visualization import make_dashboard_chat_agent

chat_agent = make_dashboard_chat_agent(
    config, ts_store=ts_store, doc_store=doc_store, knowledge_graph=kg,
)
app = create_app(config, registry, include_viz=True, chat_agent=chat_agent)

```

`create_app` mounts that agent on `/api/chat` exactly once, whether or not the plain chat endpoint is also enabled. Leave `chat_agent` off and the chat still works — it falls back to the twin’s highest-priority diagnostic agent — but a chat that draws is one argument away.

Under the hood the agent reaches for two tools, `make_chart_tool` and `make_forecast_tool`, which you can also add to a custom agent’s toolset directly when you are building your own. When the agent calls one, it returns a chart specification wrapped in markers the dashboard recognises; the chat panel lifts the chart out of the stream and renders it inline in the conversation, and **mirrors it onto the Home tab’s agent canvas** so the asset’s visuals accumulate in one place, separate from the chat transcript. `make_chart_tool` plots recent history for the fields the user named; `make_forecast_tool` projects a field forward using the forecasting backend from chapter 05. The agent decides when a chart answers the question better than words.

It can listen and speak. Set `voice_enabled = True` on the spec and the chat grows a microphone button. By default the voice path runs entirely in the browser using the Web Speech API — speech-to-text for what the operator says, text-to-speech for the twin’s reply — so it needs no server-side install and no extra dependency. If you want server-side transcription instead (for browsers without Web Speech, or to control the model), install the `voice` extra and register a `VoiceProvider`; the `/api/viz/voice/*` endpoints then handle it, and return a clear “not enabled” response when no provider is registered.

Watching the agents work

The reasoning tier (chapter 08) runs a multi-agent system: each agent observes, reasons over the live data, and acts on its own interval. The dashboard surfaces that activity natively. When a multi-agent system is attached — passed to `mount_visualization(mas=...)`, or discovered from the service registry — the **Agents** tab polls `/api/viz/agents` and renders a live card per agent showing, for each one:

- its name, domain, and the severity of its latest assessment,
- the action it decided on (and a flag when it found an anomaly),
- a short summary of its reasoning,
- the tool calls it made, with their inputs and (truncated) outputs.

This is generic: any twin with a `MultiAgentSystem` gets the Agents tab with no extra work, because the endpoint reads each agent’s latest observe→reason→act snapshot from the system. It turns the agents from a background process into something an operator can watch and trust.

The 3D viewport

For an asset you have a 3D model of, the dashboard shows that model on the Home tab and drives it from live readings. The `scene` panel is built from the same field bindings as everything else, so a hotspot on the model inherits the field’s unit and thresholds and lights up on exactly the same warn/critical logic as its KPI tile. You build a scene from the config and add it as a panel:

```

from dyon.visualization import scene_from_config, derive_default_spec
from dyon.visualization.schema import PanelSpec

spec = derive_default_spec(config)
scene = scene_from_config(
    config,
    model_url="/static/pump.glb",
    positions={"bearing_temp_c": "0m 0.4m 0m"}, # where each hotspot sits
    stress_field="bearing_temp_c",             # drives the live colour wash
)
spec.scene_enabled = True
spec.panels.insert(0, PanelSpec(
    id="model", kind="scene", title="Pump", span=2, config=scene.model_dump(),
))

mount_visualization(app, config, registry, spec=spec)

```

The model responds to the asset's state. Name a `stress_field` and the viewport applies a live colour wash to the model that ramps from healthy to stressed as that field moves between its bounds — a pump casing reddening as its bearing heats, a turbine blade darkening as fouling builds. When the field carries no thresholds of its own (a normalised wear index, say), set `stress_warn / stress_crit / stress_direction` on the scene to fix the ramp. If you have distinct models for distinct conditions, `stage_models` swaps the whole model at chosen thresholds. Hotspot labels track their fields' values, and only hotspots you have given a position are drawn in 3D, so labels never pile up.

The viewport is **compute-gated by design**. 3D rendering is the one part of the dashboard that asks something of the viewer's machine, so the client checks for WebGL2 before it loads the 3D renderer. On a machine that can run it, the operator gets the live model. On one that cannot — a low-powered tablet on a plant floor, say — the panel falls back to a 2D schematic you supply as `fallback_svg`, bound to the same fields, so the information is never lost. You can also run with no 3D model at all (`model_url=None`) and rely on the schematic alone, which is often the right call for an asset with no meaningful geometry.

A note on fidelity: a colour wash and a stage swap make the model reflect the asset's state, but a model that physically deforms with the asset — a belt that visibly slackens, a structure that flexes — needs a rigged asset with the corresponding morph targets or animations, which you supply and drive through the same `stress_field`. The framework gives you the live binding; the realism of the geometry is a property of the model you bring.

Combined twins: one dashboard for many

Chapter 10 showed how twins combine — an **aggregate** rolls a fleet up, a **collection** compares peers, a **composite** nests twins under boundary conditions, and a **network** wires them by typed relationships. The dashboard has a first-class view for each. A combined dashboard carries, alongside the usual fields, the **members** that make it up and the **topology** between them, and the client renders an **Overview** tailored to the combination:

- **aggregate** → a fleet roll-up (merged mean/min/max KPIs and average health),
- **collection** → a peer comparison (health ranking and statistical outliers),
- **composite** → the hierarchy and a boundary-condition flow diagram,
- **network** → the typed relationship graph.

Every type also shows a live status card per member; the sidebar lists the members, and selecting one **drills into that member's own full dashboard** — its tabs, charts, agents, and 3D scene — federated straight from that member's API. You build the combined spec from the members and topology:

```
from dyon.visualization import (
    derive_combined_spec, MemberRef, TopologyEdge,
)

spec = derive_combined_spec(
    combination="composite",
    asset_id="station-01", asset_name="Power Station",
    members=[
        MemberRef(id="boiler", name="Boiler", asset_type="boiler", api_base="/
boiler"),
        MemberRef(id="turbine", name="Turbine", asset_type="turbine", api_base="/
turbine"),
    ],
    hierarchy={"station-01": ["boiler", "turbine"]},
    edges=[TopologyEdge(source="boiler", target="turbine", kind="flow",
                        label="steam flow")],
)
```

If the combined twin is a real collection twin (chapter 10), `combined_spec_from_twin(twin, member_api_bases=...)` reads its members, hierarchy, boundaries, and relationships for you.

Where does a combined dashboard run? Each member is a twin that serves its own `/api/viz/*`, and the combined client federates them in the browser. That gives you two clean options. The first is to serve each member as its own process and point a thin combined launcher at their URLs — `create_combined_dashboard_app(spec)` serves the overview and federates members over CORS, holding no state itself. The second, when the combination is itself a real twin with its own agents — a composite overseer, say — is to **bundle** everything in one process: serve the overseer's own dashboard, agents, and chat at the root, and mount each member under its own path with `mount_visualization(..., base_path="/boiler")` so the browser federates them same-origin. In the bundled case the overseer's own multi-agent system shows up in the Overview's Agents tab, exactly as a single twin's does. Neither arrangement is privileged: a composite twin can be served from one process during development and split across several in production without the dashboard, or its spec, changing at all.

Optional backends

Two dashboard features lean on optional dependencies, and both degrade cleanly when the dependency is absent:

Feature	Install	Without it
Forecast chart / tool	<code>dyon[forecast]</code>	the forecast endpoint returns “not enabled”
Server-side voice	<code>dyon[voice]</code>	voice falls back to the browser's Web Speech

The dashboard client itself, the live charts, the chat panel, the agents view, and the 3D viewport need nothing beyond the framework — they render in the browser from pinned CDN libraries, so there is no

build step and no front-end toolchain to install. The `/api/viz/capabilities` endpoint reports which optional features are live, and the dashboard hides or disables what is not available rather than failing.

Where this leaves you

The dashboard is the framework's account of itself made visible. Every panel is driven by something an earlier chapter built — sensor fields become tiles, charts, and 3D hotspots; the event bus drives the live updates; the control tier's thresholds become alarms and the model's colour wash; the diagnostic agent answers in the chat and draws on the canvas; the multi-agent system shows its work in the Agents tab; and combined twins compose all of it into one federated view. You did not write a front-end to get any of it; you declared an asset, and the framework drew it.